

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-ab67b20995535f4fc.jsonl`  
- SHA-256: `15221bbb4f5161546f286ee51e1e7be8083789ca75d2a372aed64ff7ee69fd8d`  
- Source modified: `2026-07-06T06:05:57+00:00`  
- Imported at: `2026-07-08T16:00:12+00:00`  
- project: `wf_eb99bd9f-440`  
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T06:02:53.382Z)

You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite; use db.__apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest 434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which is NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow trigger), medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

DIMENSION: backwards-compat shim correctness & provider-neutrality leaks.

Investigate specifically:

- _content_item_to_video / get_video_by_* / list_videos_for_user cast provider_message_id, document_id, and provider_source_id via int(...). For Telegram they're numeric, but is there any path where these are non-numeric or None and int() raises? (e.g. a content_item created by non-shim code.)
 - mark_forwarded is a no-op: trace delivery.py ? does anything rely on forwarded state to avoid double delivery across process restarts? Is "forwards are idempotent" actually true, or can a restart re-forward matched-but-unforwarded items? (matched is set at insert BEFORE delivery.)
 - insert_video upserts a source with watched=True every time; get_channel/get_video return shapes;
 - None-handling in the get_video_by_document_id/fingerprint -> get_source_by_id chains.
- Return findings per schema.

2. assistant (2026-07-06T06:02:55.917Z)

I'll review PR #35 focusing on backwards-compat shim correctness and provider-neutrality leaks. Let me start by reading the changed files.

3. user (2026-07-06T06:02:57.041Z)

```
1      """Pydantic data models for users, channels, videos, and filter config.
2
3      These models are the in-memory representation used by the repository layer
4      (`db.py`) and the filter engine (`filters/`). They are deliberately
5      decoupled from Telethon's own types so the filter engine stays pure and
6      unit-testable without a live Telegram connection.
7      """
8
9      from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33
34
35     class Account(BaseModel):
36         """A product identity ? the owner of config, channels, routes, and content.
37
38         Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39         is the bridge used by the Bot API front-door (`from.id`) and by the
40         legacy userbot path; it is nullable so future providers that don't have a
41         Telegram id (e.g. WhatsApp) can still own an ``Account``.
42         """
43
44         model_config = ConfigDict(frozen=True)
45
46         id: int = Field(description="Stable surrogate PK; the owner key for all downstream
47         tables")
48         telegram_id: int | None = Field(
49             default=None,
50             description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
51             set.",
52         )
53         provider: str = Field(default="telegram", description="Provider namespace; future:
54         'whatsapp'")
55         display_name: str | None = None
56         created_at: datetime = Field(default_factory=_utcnow)
```

```

56 class TelegramSession(BaseModel):
57     """An optional MProto (Telethon) session linked to an :class:`Account`.
58
59     A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60     that is what makes them structurally invisible to ``manager.run``
61     (ADR 0010). ``session_path`` is data-driven so the file-location
62     convention is explicit rather than derived from the PK at runtime.
63     """
64
65     model_config = ConfigDict(frozen=True)
66
67     account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69     session_path: str = Field(description="File-backed Telethon session path
(user_id.session)")
70     enabled: bool = Field(
71         default=True,
72         description="If False, the manager skips this session (not the account)",
73     )
74     created_at: datetime = Field(default_factory=_utcnow)
75     last_login_at: datetime | None = None
76
77
78 class User(BaseModel):
79     """Deprecated: legacy v1 model conflating identity + MProto session.
80
81     Retained as a backwards-compatible view over the new ``accounts`` +
82     ``telegram_sessions`` tables (ADR 0010). New code should use
83     :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84     maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85     and tests keep working without a flag day.
86     """
87
88     model_config = ConfigDict(frozen=True)
89
90     id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
92     enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93     created_at: datetime = Field(default_factory=_utcnow)
94
95
96 class Channel(BaseModel):
97     """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99     Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100     code should use :class:`Source` directly; the ``db`` layer maps
101     ``Channel`` ? ``Source`` so existing callers and tests keep working.
102     """
103
104     model_config = ConfigDict(frozen=True)
105
106     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107     user_id: int
108     channel_id: int = Field(description="Telegram channel/chat id")
109     title: str | None = None
110     seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113 # --- provider-neutral domain models (ADR 0011, Track B) -----
114 #

```

```

115 # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116 # shapes so the schema no longer leaks Telegram-specific ids into every future
117 # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118 # make cross-provider collisions structurally impossible. See ADR 0011 for the
119 # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120 # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123 class Source(BaseModel):
124     """A configured place we ingest content from (ADR 0011).
125
126     Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127     providers add their own source kinds). ``provider_source_id`` is the
128     provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129     """
130
131     model_config = ConfigDict(frozen=True)
132
133     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134     account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135     provider: str = Field(default="telegram", description="Provider namespace")
136     provider_source_id: str = Field(
137         description="Provider-native source id (Telegram channel_id, ?)"
138     )
139     title: str | None = None
140     kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141     watched: bool = Field(
142         default=False,
143         description="ADR 0009 selective scanning: ingest from this source iff True",
144     )
145     seen_first_at: datetime = Field(default_factory=utcnow)
146     last_seen_at: datetime | None = None
147
148
149 class ContentItem(BaseModel):
150     """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152     Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153     (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154     NULL until later tracks populate them (avoids a future ``ALTER TABLE``).
155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157     """
158
159     model_config = ConfigDict(frozen=True)
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' | 'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts (Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None

```

```

177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=_utcnow)
184
185
186 class Policy(BaseModel):
187     """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189     Generalizes the legacy per-account ``filter_config`` row so one account can
190     own multiple policies (enables ADR 0009's per-route filter overrides) and so
191     future enrichment/security stages grow into the same ``config_json`` column.
192     Exactly one policy per account carries ``is_default=True``; the legacy
193     ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194     """
195
196     model_config = ConfigDict(frozen=True)
197
198     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199     account_id: int = Field(description="FK ? accounts.id")
200     name: str = Field(description="Policy name; 'default' for the account default
policy")
201     config_json: str = Field(
202         description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203     )
204     is_default: bool = Field(
205         default=False, description="Exactly one per account (UNIQUE partial index)"
206     )
207     created_at: datetime = Field(default_factory=_utcnow)
208     updated_at: datetime = Field(default_factory=_utcnow)
209
210
211 class FilterConfig(BaseModel):
212     """Per-user filter configuration.
213
214     v1 rules: duration bounds, minimum height (resolution), and max file size.
215     Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
216     new field later is backwards-compatible.
217     """
218
219     # Duration bounds in seconds. None means unbounded on that side.
220     min_duration_s: int | None = Field(default=5, ge=0)
221     max_duration_s: int | None = Field(default=7200, ge=0)
222
223     # Resolution: a video passes if its height >= target_height.
224     # None disables the resolution rule.
225     target_height: int | None = Field(default=1080, ge=1)
226
227     # Max file size in mebibytes. None disables the size rule.
228     max_size_mb: int | None = Field(default=2048, ge=1)
229
230     # Content-type exclusions. GIFs and round videos are excluded by default
231     # (their duration/dimensions are usually incidental to real video filters);
232     # silent videos are off by default since some legit clips have no audio.
233     # Existing users' stored configs are unaffected ? the DB JSON blob overrides
234     # these defaults, so this only changes the behaviour for fresh installs.
235     exclude_gif: bool = Field(
236         default=True,
237         description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
238     )
239     exclude_round: bool = Field(

```

```

240         default=True,
241         description="If True, exclude circular video messages (round_message=True)",
242     )
243     exclude_nosound: bool = Field(
244         default=False,
245         description="If True, exclude videos without an audio track (nosound=True)",
246     )
247
248     # How matched videos are delivered: forward the video, send a link, or both.
249     # Stored in the same JSON blob, so adding this field is backwards-compatible
250     # (existing rows without it default to FORWARD).
251     delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
252
253     # Where matched videos / link messages are sent. Accepts:
254     # - "me" (default): Saved Messages ? exact v1.1 behaviour.
255     # - "@username": a public channel the account is a member of.
256     # - "-100..." or a bare int-as-string: a numeric Telegram chat id
257     #   (private channels without a username).
258     # The string is passed verbatim to Telethon's forward_messages / send_message.
259     # See ADR 0005 for rationale and access requirements.
260     delivery_target: str = Field(
261         default="me",
262         min_length=1,
263         description="Telethon entity to deliver to: 'me', '@username', or a chat id",
264     )
265
266     @field_validator("delivery_target", mode="before")
267     @classmethod
268     def _coerce_delivery_target(cls, v: object) -> str:
269         """Accept int chat ids (e.g. -1001234567890) and coerce to str.
270
271         Telethon accepts both str and int as an entity, but storing as str keeps
272         the JSON blob uniform and lets '@username' / 'me' / numeric ids share
273         one field type.
274         """
275         if v is None:
276             return "me"
277         if isinstance(v, int):
278             return str(v)
279         return str(v)
280
281     @model_validator(mode="after")
282     def _check_duration_bounds(self) -> FilterConfig:
283         """Ensure min_duration_s <= max_duration_s when both are set."""
284         if (
285             self.min_duration_s is not None
286             and self.max_duration_s is not None
287             and self.min_duration_s > self.max_duration_s
288         ):
289             raise ValueError(
290                 f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
291                 f"({self.max_duration_s})"
292             )
293         return self
294
295     class VideoRecord(BaseModel):
296         """A seen video message + its metadata. Drives dedup and future /search.
297
298         Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
299         New code should use :class:`ContentItem`; the ``db`` layer maps
300         ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
301         """
302
303         model_config = ConfigDict(frozen=True)

```

```

304
305     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
306     user_id: int
307     channel_id: int = Field(description="Origin Telegram channel id")
308     message_id: int = Field(description="Telegram message id within the channel")
309     document_id: int | None = Field(
310         default=None,
311         description="Telegram document.id ? reused across forwards; Tier-1 content
dedup",
312     )
313
314     # Metadata extracted from the Telethon message (may be None if absent).
315     duration_s: int | None = None
316     width: int | None = None
317     height: int | None = None
318     size_bytes: int | None = None
319     mime_type: str | None = None
320
321     matched: bool = False
322     forwarded: bool = False
323     seen_at: datetime = Field(default_factory=utcnow)
324
325     @property
326     def dedup_key(self) -> tuple[int, int, int]:
327         """The natural key used for deduplication in the ``videos`` table."""
328         return (self.user_id, self.channel_id, self.message_id)
329
330     @property
331     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
332         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
333
334         Returns None if any field is missing (cannot fingerprint).
335         """
336         if None in (self.size_bytes, self.duration_s, self.height, self.width):
337             return None
338         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
339
340
341     # Default filter config shared by the bootstrap onboarding step.
342     DEFAULT_FILTER_CONFIG = FilterConfig()
343

```

4. user (2026-07-06T06:02:57.351Z)

```

1     """SQLite persistence layer (aiosqlite).
2
3     Schema, migrations, and repository functions. See ``docs/design.md`` §3 for
4     the legacy data model and ``docs/adr/0011-provider-neutral-domain-schema.md``
5     for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6     ``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
7
8     The repository functions take an ``aiosqlite.Connection`` so tests can inject an
9     in-memory database (``aiosqlite.connect(":memory:")``).
10    """
11
12    from __future__ import annotations
13
14    from collections.abc import Awaitable, Callable, Iterable
15    from datetime import UTC, datetime
16
17    import aiosqlite
18
19    from tg_video_filter.models import (

```

```

20     Account,
21     Channel,
22     ContentItem,
23     FilterConfig,
24     Policy,
25     Source,
26     TelegramSession,
27     User,
28     VideoRecord,
29 )
30
31 # --- DDL -----
32
33 _SCHEMA = ""
34 CREATE TABLE IF NOT EXISTS accounts (
35     id            INTEGER PRIMARY KEY,
36     telegram_id   INTEGER UNIQUE,
37     provider      TEXT NOT NULL DEFAULT 'telegram',
38     display_name  TEXT,
39     created_at    TEXT NOT NULL
40 );
41
42 CREATE TABLE IF NOT EXISTS telegram_sessions (
43     account_id    INTEGER PRIMARY KEY REFERENCES accounts(id),
44     phone         TEXT,
45     session_path  TEXT NOT NULL,
46     enabled       INTEGER NOT NULL DEFAULT 1,
47     created_at    TEXT NOT NULL,
48     last_login_at TEXT
49 );
50
51 CREATE TABLE IF NOT EXISTS channels (
52     id            INTEGER PRIMARY KEY AUTOINCREMENT,
53     user_id       INTEGER NOT NULL REFERENCES accounts(id),
54     channel_id    INTEGER NOT NULL,
55     title         TEXT,
56     seen_first_at TEXT NOT NULL,
57     UNIQUE (user_id, channel_id)
58 );
59
60 CREATE TABLE IF NOT EXISTS videos (
61     id            INTEGER PRIMARY KEY AUTOINCREMENT,
62     user_id       INTEGER NOT NULL REFERENCES accounts(id),
63     channel_id    INTEGER NOT NULL,
64     message_id    INTEGER NOT NULL,
65     document_id   INTEGER,
66     duration_s    INTEGER,
67     width         INTEGER,
68     height        INTEGER,
69     size_bytes    INTEGER,
70     mime_type     TEXT,
71     matched       INTEGER NOT NULL DEFAULT 0,
72     forwarded     INTEGER NOT NULL DEFAULT 0,
73     seen_at       TEXT NOT NULL,
74     UNIQUE (user_id, channel_id, message_id)
75 );
76
77 CREATE TABLE IF NOT EXISTS filter_config (
78     user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
79     config_json   TEXT NOT NULL
80 );
81
82 -- Provider-neutral domain tables (ADR 0011, Track B). ``sources``,
83 -- ``content_items``, and ``policies`` are populated by migration 003 (and by
84 -- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``

```



```

85 -- are created empty here so the schema is complete ? Track C (#28) implements
86 -- the delivery logic that populates them.
87
88 CREATE TABLE IF NOT EXISTS sources (
89     id                INTEGER PRIMARY KEY AUTOINCREMENT,
90     account_id        INTEGER NOT NULL REFERENCES accounts(id),
91     provider          TEXT NOT NULL DEFAULT 'telegram',
92     provider_source_id TEXT NOT NULL,
93     title             TEXT,
94     kind              TEXT NOT NULL DEFAULT 'channel',
95     watched           INTEGER NOT NULL DEFAULT 0,
96     seen_first_at     TEXT NOT NULL,
97     last_seen_at      TEXT,
98     UNIQUE (account_id, provider, provider_source_id)
99 );
100
101 CREATE TABLE IF NOT EXISTS content_items (
102     id                INTEGER PRIMARY KEY AUTOINCREMENT,
103     account_id        INTEGER NOT NULL REFERENCES accounts(id),
104     source_id         INTEGER NOT NULL REFERENCES sources(id),
105     provider          TEXT NOT NULL DEFAULT 'telegram',
106     provider_message_id TEXT NOT NULL,
107     content_type       TEXT NOT NULL DEFAULT 'video',
108     text              TEXT,
109     link_url          TEXT,
110     media_ref         TEXT,
111     document_id       TEXT,
112     duration_s        INTEGER,
113     width             INTEGER,
114     height            INTEGER,
115     size_bytes        INTEGER,
116     mime_type         TEXT,
117     dedup_key         TEXT NOT NULL,
118     matched           INTEGER NOT NULL DEFAULT 0,
119     seen_at           TEXT NOT NULL,
120     UNIQUE (account_id, source_id, provider_message_id),
121     UNIQUE (account_id, dedup_key)
122 );
123
124 CREATE TABLE IF NOT EXISTS policies (
125     id                INTEGER PRIMARY KEY AUTOINCREMENT,
126     account_id        INTEGER NOT NULL REFERENCES accounts(id),
127     name              TEXT NOT NULL,
128     config_json       TEXT NOT NULL,
129     is_default        INTEGER NOT NULL DEFAULT 0,
130     created_at        TEXT NOT NULL,
131     updated_at        TEXT NOT NULL,
132     UNIQUE (account_id, name)
133 );
134
135 CREATE TABLE IF NOT EXISTS destinations (
136     id                INTEGER PRIMARY KEY AUTOINCREMENT,
137     account_id        INTEGER NOT NULL REFERENCES accounts(id),
138     provider          TEXT NOT NULL DEFAULT 'telegram',
139     target            TEXT NOT NULL,
140     kind              TEXT NOT NULL DEFAULT 'saved',
141     title             TEXT,
142     is_output         INTEGER NOT NULL DEFAULT 0,
143     created_at        TEXT NOT NULL,
144     UNIQUE (account_id, provider, target)
145 );
146
147 CREATE TABLE IF NOT EXISTS routing_rules (
148     id                INTEGER PRIMARY KEY AUTOINCREMENT,
149     account_id        INTEGER NOT NULL REFERENCES accounts(id),

```

```

150     source_id      INTEGER NOT NULL REFERENCES sources(id),
151     policy_id      INTEGER REFERENCES policies(id),
152     destination_id INTEGER NOT NULL REFERENCES destinations(id),
153     enabled        INTEGER NOT NULL DEFAULT 1,
154     priority       INTEGER NOT NULL DEFAULT 0,
155     created_at     TEXT NOT NULL,
156     UNIQUE (account_id, source_id, destination_id)
157 );
158
159 CREATE TABLE IF NOT EXISTS delivery_attempts (
160     id              INTEGER PRIMARY KEY AUTOINCREMENT,
161     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
162     destination_id  INTEGER NOT NULL REFERENCES destinations(id),
163     route_id       INTEGER REFERENCES routing_rules(id),
164     status         TEXT NOT NULL DEFAULT 'pending',
165     attempt_count   INTEGER NOT NULL DEFAULT 0,
166     last_error      TEXT,
167     created_at     TEXT NOT NULL,
168     delivered_at    TEXT,
169     next_retry_at   TEXT,
170     UNIQUE (content_item_id, destination_id)
171 );
172 """"
173
174 # Note: the ``idx_videos_user_document`` partial unique index is created by
175 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
176 # which is present on fresh tables but only added to legacy v1 DBs by the
177 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
178 # DBs that don't yet have the column.
179
180
181 async def init_db(db_path: str) -> None:
182     """Open the database at ``db_path`` and create/migrate the schema.
183
184     Safe to call on every startup. Applies the base schema (``IF NOT EXISTS``)
185     then runs any pending migrations via the ``user_version`` ladder.
186     """
187     async with aiosqlite.connect(db_path) as conn:
188         await _apply_schema(conn)
189
190
191 async def _apply_schema(conn: aiosqlite.Connection) -> None:
192     # Enable WAL mode (concurrent reads+writes), foreign key enforcement,
193     # and a busy timeout so concurrent access waits instead of failing immediately.
194     await conn.execute("PRAGMA journal_mode = WAL")
195     await conn.execute("PRAGMA foreign_keys = ON")
196     await conn.execute("PRAGMA busy_timeout = 5000")
197     await conn.executescript(_SCHEMA)
198     # ADR 0011 housekeeping: ``_SCHEMA`` still creates the legacy
199     # ``channels`` / ``videos`` / ``filter_config`` tables (they're needed for
200     # migration 002's FK rebuild and migration 003's backfill on legacy DBs).
201     # Once migration 003 has run (signalled by ``channels_deprecated`` existing),
202     # any *empty* recreations of those legacy tables from a subsequent
203     # ``init_db`` are dropped here so they don't shadow the renamed copies.
204     if await _table_exists(conn, "channels_deprecated"):
205         await conn.execute("DROP TABLE IF EXISTS channels")
206         await conn.execute("DROP TABLE IF EXISTS videos")
207         await conn.execute("DROP TABLE IF EXISTS filter_config")
208     await _apply_migrations(conn)
209     await conn.commit()
210
211
212 # --- migrations -----
213 #
214 # Migrations are applied in order via SQLite's ``PRAGMA user_version`` counter.

```

```

215 # Each migration is idempotent and bumps the version by 1. ``_SCHEMA`` above is
216 # the source of truth for *fresh* databases; the migrations below reconcile
217 # *existing* databases that were created by an older schema version.
218 #
219 # To add a new migration:
220 # 1. Append a ``(version, _migration_NN)`` tuple to ``_MIGRATIONS``.
221 # 2. Implement ``async def _migration_NN(conn) -> None`` (must be idempotent).
222 # 3. Add a test in ``tests/test_db.py`` that seeds an old schema, runs
223 #    ``init_db``, and asserts the new structure.
224
225 _MIGRATIONS: list[tuple[int, Callable[[aiosqlite.Connection], Awaitable[None]]]] = []
226
227
228 def _migration(
229     version: int,
230 ) -> Callable[
231     [Callable[[aiosqlite.Connection], Awaitable[None]]],
232     Callable[[aiosqlite.Connection], Awaitable[None]],
233 ]:
234     """Register a migration that brings the DB up to ``user_version == version``."""
235
236     def decorator(
237         fn: Callable[[aiosqlite.Connection], Awaitable[None]],
238     ) -> Callable[[aiosqlite.Connection], Awaitable[None]]:
239         _MIGRATIONS.append((version, fn))
240         return fn
241
242     return decorator
243
244
245 # --- provider-neutral dedup key (ADR 0011, Option 1A) -----
246 #
247 # Pure function: computes the ``content_items.dedup_key`` value from metadata
248 # already extracted at ingestion time (no download ? honors ADR 0003). The
249 # decision tree is ordered strongest-key first; each branch is guarded so
250 # partial metadata can never collapse to a degenerate key (e.g. ``tg:fp:0:0:0:0``).
251 # See ``compute_dedup_key_test`` in ``tests/test_db.py`` for the full matrix.
252
253
254 def compute_dedup_key(
255     *,
256     provider: str,
257     document_id: str | None = None,
258     source_id: int | str | None = None,
259     provider_message_id: str | None = None,
260     size_bytes: int | None = None,
261     duration_s: int | None = None,
262     height: int | None = None,
263     width: int | None = None,
264 ) -> str:
265     """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.
266
267     Implements the ADR 0011 Option 1A decision tree:
268
269     - **Telegram** (``provider == "telegram"``):
270       1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
271       2. Else if **all four** fingerprint fields (size, duration, height, width)
272          are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273       3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274          ``source_id`` is required because Telegram ``message_id`` is only
275          unique within a channel).
276     - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
277       ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278       providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279

```

```

280     Raises ``ValueError`` if the fallback branch is reached without the fields
281     it requires (``source_id`` + ``provider_message_id``) ? a degenerate
282     ``dedup_key`` is worse than a loud failure at ingestion time.
283     """
284     p = provider.lower()
285     if p == "telegram":
286         if document_id is not None and document_id != "":
287             return f"tg:doc:{document_id}"
288         # All four fingerprint fields are required to avoid a degenerate
289         # ``tg:fp:0:0:0:0`` key when some metadata is missing.
290         if None not in (size_bytes, duration_s, height, width):
291             return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292         # Tier-0 fallback.
293         if source_id is None or provider_message_id is None:
294             raise ValueError(
295                 "Telegram dedup_key fallback requires source_id and provider_message_id
296                 "
297             )
298         return f"tg:msg:{source_id}:{provider_message_id}"
299     # Future-proof default for providers without a dedicated branch.
300     if source_id is None or provider_message_id is None:
301         raise ValueError(
302             f"dedup_key for provider={provider!r} requires source_id and
303             provider_message_id"
304         )
305     return f"{p}:msg:{source_id}:{provider_message_id}"
306
307 async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308     """Run every registered migration whose version > current ``user_version``."""
309     async with conn.execute("PRAGMA user_version") as cur:
310         row = await cur.fetchone()
311         current = int(row[0]) if row else 0
312
313     for version, fn in _MIGRATIONS:
314         if version <= current:
315             continue
316         await fn(conn)
317         await conn.execute(f"PRAGMA user_version = {version}")
318
319
320 async def _column_exists(conn: aiosqlite.Connection, table: str, column: str) -> bool:
321     """Return True if ``column`` exists on ``table`` (async, PRAGMA table_info)."""
322     async with conn.execute(f"PRAGMA table_info({table})") as cur:
323         rows = await cur.fetchall()
324         return any(row[1] == column for row in rows)
325
326
327 @_migration(1)
328 async def _migration_001_add_document_id(conn: aiosqlite.Connection) -> None:
329     """Add ``document_id`` column + partial unique index (ADR 0006 content dedup).
330
331     Idempotent: safe on fresh DBs (column/index already present) and on v1 DBs
332     created before this migration existed.
333     """
334     if not await _column_exists(conn, "videos", "document_id"):
335         await conn.execute("ALTER TABLE videos ADD COLUMN document_id INTEGER")
336     # Create the partial unique index for both fresh and migrated DBs. Uses
337     # IF NOT EXISTS so it is safe on DBs where it was already created.
338     await conn.execute(
339         "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
340         "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
341     )
342

```

```

343
344 @migration(2)
345 async def _migration_002_identity_split(conn: aiosqlite.Connection) -> None:
346     """Split the legacy ``users`` table into ``accounts`` + ``telegram_sessions``.
347
348     ADR 0010. Integer-preserving: ``accounts.id`` copies ``users.id`` and the
349     data in ``channels`` / ``videos`` / ``filter_config`` is copied verbatim,
350     so their integer ``user_id`` values still resolve to the same identity (now
351     via ``accounts.id``). The legacy ``users`` table is renamed to
352     ``users_deprecated`` and kept for one release as a rollback safety net; a
353     follow-up migration drops it.
354
355     FK contract: ``channels`` / ``videos`` / ``filter_config`` are rebuilt with
356     ``REFERENCES accounts(id)`` so migrated DBs end up with the *same* FK shape
357     as fresh DBs. (A naive rename of ``users`` would leave SQLite rewriting
358     those FKs to point at ``users_deprecated``, diverging from fresh installs
359     and breaking inserts for new accounts ? see PR #33 review.)
360
361     Idempotent: a no-op once the legacy ``users`` table has been renamed.
362     """
363     # If the legacy users table is gone, this migration has already run
364     # (or the DB is fresh and only has accounts). Nothing to do.
365     if not await _table_exists(conn, "users"):
366         return
367
368     # Build accounts from the legacy users table. accounts.id := users.id.
369     # (The empty accounts table created by _SCHEMA on a legacy DB is dropped
370     # and recreated here so the INSERT can populate it cleanly.)
371     await conn.execute("DROP TABLE IF EXISTS accounts")
372     await conn.execute(
373         """
374         CREATE TABLE accounts (
375             id            INTEGER PRIMARY KEY,
376             telegram_id   INTEGER UNIQUE,
377             provider      TEXT NOT NULL DEFAULT 'telegram',
378             display_name  TEXT,
379             created_at    TEXT NOT NULL
380         )
381         """
382     )
383     await conn.execute(
384         """
385         INSERT INTO accounts (id, telegram_id, provider, created_at)
386         SELECT id, id, 'telegram', created_at FROM users
387         """
388     )
389
390     # Build telegram_sessions, one row per legacy userbot user.
391     # (Same drop-and-recreate as accounts: _SCHEMA created an empty copy.)
392     await conn.execute("DROP TABLE IF EXISTS telegram_sessions")
393     await conn.execute(
394         """
395         CREATE TABLE telegram_sessions (
396             account_id   INTEGER PRIMARY KEY REFERENCES accounts(id),
397             phone        TEXT,
398             session_path TEXT NOT NULL,
399             enabled      INTEGER NOT NULL DEFAULT 1,
400             created_at   TEXT NOT NULL,
401             last_login_at TEXT
402         )
403         """
404     )
405     # session_path is built from the *telegram_id* (= old users.id) to match
406     # the convention telethon_client.build_client used pre-migration. The
407     # sessions_dir prefix is prepended at read time (Settings.sessions_dir)

```

```

408     # so this migration does not need to know the runtime path.
409     await conn.execute(
410         """
411         INSERT INTO telegram_sessions (account_id, phone, session_path, enabled,
created_at)
412         SELECT id, phone, 'user_' || id || '.session', enabled, created_at
413         FROM users
414         """
415     )
416
417     # Rebuild channels / videos / filter_config so their FK constraints point
418     # at accounts(id) instead of the legacy users(id). SQLite's
419     # ALTER TABLE RENAME rewrites FK references in *other* tables to follow
420     # the rename ? so a naive rename of users ? users_deprecated would leave
421     # these tables referencing users_deprecated(id), diverging from fresh DBs
422     # (which reference accounts(id)) and breaking inserts for new accounts.
423     # The rebuild: rename the data-holding legacy table out of the way, create
424     # the new one with the correct FK, copy the data back, drop the renamed copy.
425     for table, new_ddl, copy_cols in (
426         (
427             "channels",
428             """
429             CREATE TABLE channels (
430                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
431                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
432                 channel_id    INTEGER NOT NULL,
433                 title         TEXT,
434                 seen_first_at TEXT NOT NULL,
435                 UNIQUE (user_id, channel_id)
436             )
437             """,
438             "id, user_id, channel_id, title, seen_first_at",
439         ),
440         (
441             "videos",
442             """
443             CREATE TABLE videos (
444                 id            INTEGER PRIMARY KEY AUTOINCREMENT,
445                 user_id       INTEGER NOT NULL REFERENCES accounts(id),
446                 channel_id    INTEGER NOT NULL,
447                 message_id    INTEGER NOT NULL,
448                 document_id   INTEGER,
449                 duration_s    INTEGER,
450                 width         INTEGER,
451                 height        INTEGER,
452                 size_bytes    INTEGER,
453                 mime_type     TEXT,
454                 matched       INTEGER NOT NULL DEFAULT 0,
455                 forwarded     INTEGER NOT NULL DEFAULT 0,
456                 seen_at       TEXT NOT NULL,
457                 UNIQUE (user_id, channel_id, message_id)
458             )
459             """,
460             "id, user_id, channel_id, message_id, document_id, duration_s, width, "
461             "height, size_bytes, mime_type, matched, forwarded, seen_at",
462         ),
463         (
464             "filter_config",
465             """
466             CREATE TABLE filter_config (
467                 user_id       INTEGER PRIMARY KEY REFERENCES accounts(id),
468                 config_json   TEXT NOT NULL
469             )
470             """,
471             "user_id, config_json",

```

```

472         ),
473     ):
474         await conn.execute(f"ALTER TABLE {table} RENAME TO {table}__legacy_data")
475         await conn.execute(new_ddl)
476         await conn.execute(
477             f"INSERT INTO {table} ({copy_cols}) SELECT {copy_cols} FROM
{table}__legacy_data"
478         )
479         await conn.execute(f"DROP TABLE {table}__legacy_data")
480
481         # The partial unique index for document_id dedup (created by migration 001
482         # against the legacy videos table) was dropped along with the legacy copy
483         # above. Recreate it on the new videos table.
484         await conn.execute(
485             "CREATE UNIQUE INDEX IF NOT EXISTS idx_videos_user_document "
486             "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
487         )
488
489         # Rename the legacy users table (do NOT drop ? kept as a rollback safety
490         # net for one release per ADR 0010). The data-holding channels/videos/
491         # filter_config tables were already rebuilt above with FKs to accounts(id),
492         # so renaming users no longer retargets any live FK.
493         await conn.execute("ALTER TABLE users RENAME TO users_deprecated")
494
495
496 @migration(3)
497 async def _migration_003_domain_schema(conn: aiosqlite.Connection) -> None:
498     """Land the provider-neutral domain tables (ADR 0011, Track B).
499
500     Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
501     ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
502     DBs ``_SCHEMA`` already created them; on pre-ADR-0011 DBs this creates
503     them), then backfills the three populated ones from the legacy
504     ``channels`` / ``videos`` / ``filter_config`` tables and renames the
505     legacy tables to ``*_deprecated`` (one-release rollback net, ADR 0010
506     pattern). Idempotent: a no-op once the legacy tables have been renamed.
507     """
508     # Idempotency gate: if the legacy videos table is gone, this migration has
509     # already run (or the DB is fresh and _SCHEMA's legacy tables were already
510     # dropped by _apply_schema post-003). Nothing to backfill.
511     if not await _table_exists(conn, "videos"):
512         return
513
514     # 1. Backfill sources from channels. ADR 0009 backfill: already-seen
515     # channels stay watched (watched=1). provider_source_id is the Telegram
516     # channel_id (cast to TEXT for the provider-neutral column).
517     await conn.execute(
518         """
519         INSERT OR IGNORE INTO sources
520             (account_id, provider, provider_source_id, title, kind, watched,
521              seen_first_at, last_seen_at)
522         SELECT user_id, 'telegram', CAST(channel_id AS TEXT), title, 'channel',
523                1, seen_first_at, seen_first_at
524         FROM channels
525         """
526     )
527
528     # 2. Backfill content_items from videos, computing dedup_key in pure SQL.
529     # The three-branch decision tree (Option 1A):
530     # a) document_id IS NOT NULL          ? 'tg:doc:' || document_id
531     # b) all four fingerprint fields set  ? 'tg:fp:' || size:dur:h:w
532     # c) else (Tier-0 fallback)           ? 'tg:msg:' || source_id:msg_id
533     # The ``CASE`` is ordered strongest-first and each branch is guarded so
534     # partial metadata never collapses to a degenerate key. source_id is
535     # resolved by joining the channel's provider_source_id back to sources.id

```

```

536 # (Telegram message_id is per-channel, so source_id is required).
537 await conn.execute(
538     """
539     INSERT OR IGNORE INTO content_items
540     (account_id, source_id, provider, provider_message_id, content_type,
541     document_id, duration_s, width, height, size_bytes, mime_type,
542     dedup_key, matched, seen_at)
543     SELECT
544     v.user_id,
545     s.id,
546     'telegram',
547     CAST(v.message_id AS TEXT),
548     'video',
549     CASE WHEN v.document_id IS NOT NULL THEN CAST(v.document_id AS TEXT) END,
550     v.duration_s, v.width, v.height, v.size_bytes, v.mime_type,
551     CASE
552     WHEN v.document_id IS NOT NULL
553     THEN 'tg:doc:' || v.document_id
554     WHEN v.size_bytes IS NOT NULL AND v.duration_s IS NOT NULL
555     AND v.height IS NOT NULL AND v.width IS NOT NULL
556     THEN 'tg:fp:' || v.size_bytes || ':' || v.duration_s || ':'
557     || v.height || ':' || v.width
558     ELSE 'tg:msg:' || s.id || ':' || v.message_id
559     END,
560     v.matched,
561     v.seen_at
562     FROM videos v
563     JOIN sources s
564     ON s.account_id = v.user_id
565     AND s.provider = 'telegram'
566     AND s.provider_source_id = CAST(v.channel_id AS TEXT)
567     """
568 )
569
570 # 3. Backfill policies from filter_config: one 'default' policy per account
571 # (Option 2A). is_default=1 so the legacy load_filter_config shim reads
572 # this row. UNIQUE(account_id, name) dedups idempotent re-runs.
573 await conn.execute(
574     """
575     INSERT OR IGNORE INTO policies (account_id, name, config_json, is_default,
576     created_at, updated_at)
577     SELECT user_id, 'default', config_json, 1, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP
578     FROM filter_config
579     """
580 )
581
582 # 4. Rename (don't drop) the legacy tables ? kept for one release as a
583 # rollback safety net (ADR 0010 pattern).
584 await conn.execute("ALTER TABLE channels RENAME TO channels_deprecated")
585 await conn.execute("ALTER TABLE videos RENAME TO videos_deprecated")
586 await conn.execute("ALTER TABLE filter_config RENAME TO filter_config_deprecated")
587
588
589 async def _table_exists(conn: aiosqlite.Connection, table: str) -> bool:
590     """Return True if ``table`` exists (async, PRAGMA table_list / sqlite_master)."""
591     async with conn.execute(
592         "SELECT 1 FROM sqlite_master WHERE type='table' AND name=?", (table,)
593     ) as cur:
594         return await cur.fetchone() is not None
595
596
597 # --- helpers -----
598
599
600 def _now() -> datetime:

```



```

601     """Timezone-aware UTC 'now', used for policy timestamps."""
602     return datetime.now(UTC)
603
604
605 def _row_to_account(row: aiosqlite.Row) -> Account:
606     return Account(
607         id=row["id"],
608         telegram_id=row["telegram_id"],
609         provider=row["provider"],
610         display_name=row["display_name"],
611         created_at=row["created_at"], # stored as ISO string; pydantic parses
612     )
613
614
615 def _row_to_telegram_session(row: aiosqlite.Row) -> TelegramSession:
616     return TelegramSession(
617         account_id=row["account_id"],
618         phone=row["phone"],
619         session_path=row["session_path"],
620         enabled=bool(row["enabled"]),
621         created_at=row["created_at"],
622         last_login_at=row["last_login_at"],
623     )
624
625
626 def _account_session_to_user(account: Account, session: TelegramSession | None) -> User:
627     """Project an (account, optional session) pair onto the legacy ``User`` view.
628
629     Used by the deprecated ``upsert_user``/``get_user``/``list_enabled_users``
630     shims so pre-ADR-0010 callers and tests keep working. A bot-only account
631     (no session) maps to a ``User`` with ``enabled=False`` so the legacy
632     "list enabled users" semantics exclude it ? matching the structural rule
633     that bot-only users are not runnable Telethon clients.
634     """
635     if session is None:
636         return User(
637             id=account.telegram_id if account.telegram_id is not None else account.id,
638             phone=None,
639             enabled=False,
640             created_at=account.created_at,
641         )
642     return User(
643         id=account.telegram_id if account.telegram_id is not None else account.id,
644         phone=session.phone,
645         enabled=session.enabled,
646         created_at=session.created_at,
647     )
648
649
650 def _row_to_channel(row: aiosqlite.Row) -> Channel:
651     return Channel(
652         id=row["id"],
653         user_id=row["user_id"],
654         channel_id=row["channel_id"],
655         title=row["title"],
656         seen_first_at=row["seen_first_at"],
657     )
658
659
660 def _row_to_video(row: aiosqlite.Row) -> VideoRecord:
661     return VideoRecord(
662         id=row["id"],
663         user_id=row["user_id"],
664         channel_id=row["channel_id"],
665         message_id=row["message_id"],

```

```

666         document_id=row["document_id"],
667         duration_s=row["duration_s"],
668         width=row["width"],
669         height=row["height"],
670         size_bytes=row["size_bytes"],
671         mime_type=row["mime_type"],
672         matched=bool(row["matched"]),
673         forwarded=bool(row["forwarded"]),
674         seen_at=row["seen_at"],
675     )
676
677
678 def _row_to_source(row: aiosqlite.Row) -> Source:
679     return Source(
680         id=row["id"],
681         account_id=row["account_id"],
682         provider=row["provider"],
683         provider_source_id=row["provider_source_id"],
684         title=row["title"],
685         kind=row["kind"],
686         watched=bool(row["watched"]),
687         seen_first_at=row["seen_first_at"],
688         last_seen_at=row["last_seen_at"],
689     )
690
691
692 def _row_to_content_item(row: aiosqlite.Row) -> ContentItem:
693     return ContentItem(
694         id=row["id"],
695         account_id=row["account_id"],
696         source_id=row["source_id"],
697         provider=row["provider"],
698         provider_message_id=row["provider_message_id"],
699         content_type=row["content_type"],
700         text=row["text"],
701         link_url=row["link_url"],
702         media_ref=row["media_ref"],
703         document_id=row["document_id"],
704         duration_s=row["duration_s"],
705         width=row["width"],
706         height=row["height"],
707         size_bytes=row["size_bytes"],
708         mime_type=row["mime_type"],
709         dedup_key=row["dedup_key"],
710         matched=bool(row["matched"]),
711         seen_at=row["seen_at"],
712     )
713
714
715 def _row_to_policy(row: aiosqlite.Row) -> Policy:
716     return Policy(
717         id=row["id"],
718         account_id=row["account_id"],
719         name=row["name"],
720         config_json=row["config_json"],
721         is_default=bool(row["is_default"]),
722         created_at=row["created_at"],
723         updated_at=row["updated_at"],
724     )
725
726
727 # --- account + session repos (ADR 0010) -----
728
729
730 async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:

```

```

731     """Insert or update an account row; returns the persisted Account."""
732     await conn.execute(
733         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734         VALUES (?, ?, ?, ?, ?)
735         ON CONFLICT(id) DO UPDATE SET
736             telegram_id=excluded.telegram_id,
737             provider=excluded.provider,
738             display_name=excluded.display_name""",
739         (
740             account.id,
741             account.telegram_id,
742             account.provider,
743             account.display_name,
744             account.created_at.isoformat(),
745         ),
746     )
747     await conn.commit()
748     return await get_account(conn, account.id) # type: ignore[return-value]
749
750
751     async def get_account(conn: aiosqlite.Connection, account_id: int) -> Account | None:
752         conn.row_factory = aiosqlite.Row
753         async with conn.execute("SELECT * FROM accounts WHERE id = ?", (account_id,)) as
cur:
754             row = await cur.fetchone()
755             return _row_to_account(row) if row else None
756
757
758     async def get_account_by_telegram_id(
759         conn: aiosqlite.Connection, telegram_id: int
760     ) -> Account | None:
761         """Resolve a Bot API ``from.id`` (or userbot account id) to an Account."""
762         conn.row_factory = aiosqlite.Row
763         async with conn.execute("SELECT * FROM accounts WHERE telegram_id = ?",
(telegram_id,)) as cur:
764             row = await cur.fetchone()
765             return _row_to_account(row) if row else None
766
767
768     async def upsert_telegram_session(
769         conn: aiosqlite.Connection, session: TelegramSession
770     ) -> TelegramSession:
771         """Insert or update a telegram_sessions row; returns the persisted session."""
772         await conn.execute(
773             """INSERT INTO telegram_sessions
774             (account_id, phone, session_path, enabled, created_at, last_login_at)
775             VALUES (?, ?, ?, ?, ?, ?)
776             ON CONFLICT(account_id) DO UPDATE SET
777                 phone=excluded.phone,
778                 session_path=excluded.session_path,
779                 enabled=excluded.enabled,
780                 last_login_at=excluded.last_login_at""",
781             (
782                 session.account_id,
783                 session.phone,
784                 session.session_path,
785                 int(session.enabled),
786                 session.created_at.isoformat(),
787                 session.last_login_at.isoformat() if session.last_login_at else None,
788             ),
789         )
790         await conn.commit()
791         conn.row_factory = aiosqlite.Row
792         async with conn.execute(
793             "SELECT * FROM telegram_sessions WHERE account_id = ?", (session.account_id,)

```

```

794         ) as cur:
795             row = await cur.fetchone()
796             # The row must exist ? we just upserted it by PK (account_id). Assert the
797             # invariant explicitly so mypy can narrow Row | None ? Row without a
798             # type-ignore comment masking a real future regression.
799             assert row is not None
800             return _row_to_telegram_session(row)
801
802
803 async def get_telegram_session(
804     conn: aiomysqlite.Connection, account_id: int
805 ) -> TelegramSession | None:
806     conn.row_factory = aiomysqlite.Row
807     async with conn.execute(
808         "SELECT * FROM telegram_sessions WHERE account_id = ?", (account_id,)
809     ) as cur:
810         row = await cur.fetchone()
811         return _row_to_telegram_session(row) if row else None
812
813
814 async def list_runnable_sessions(
815     conn: aiomysqlite.Connection,
816 ) -> list[tuple[Account, TelegramSession]]:
817     """Return (account, session) pairs for sessions the manager should start.
818
819     This is the ADR 0010 structural rule: ``manager.run`` iterates this list,
820     not ``accounts``. A bot-only user has no ``telegram_sessions`` row and is
821     therefore inherently invisible to the Telethon supervisor ? the "don't
822     start a client for bot-only users" invariant is enforced by query shape.
823     """
824     conn.row_factory = aiomysqlite.Row
825     async with conn.execute(
826         """SELECT a.*, s.phone AS s_phone, s.session_path, s.enabled AS s_enabled,
827             s.created_at AS s_created_at, s.last_login_at
828             FROM accounts a
829             JOIN telegram_sessions s ON s.account_id = a.id
830             WHERE s.enabled = 1"""
831     ) as cur:
832         rows = await cur.fetchall()
833         return [
834             (
835                 _row_to_account(r),
836                 TelegramSession(
837                     account_id=r["id"],
838                     phone=r["s_phone"],
839                     session_path=r["session_path"],
840                     enabled=bool(r["s_enabled"]),
841                     created_at=r["s_created_at"],
842                     last_login_at=r["last_login_at"],
843                 ),
844             )
845             for r in rows
846         ]
847
848
849 # --- legacy User shims (backwards-compatible view over accounts + sessions) --
850
851
852 async def upsert_user(conn: aiomysqlite.Connection, user: User) -> User:
853     """Deprecated shim: upsert the (account, session) pair implied by a ``User``.
854
855     Maps the legacy v1 ``User`` onto the ADR 0010 schema: ``User.id`` becomes
856     both ``accounts.id`` and ``accounts.telegram_id`` (legacy users are
857     Telegram userbot accounts), and a ``telegram_sessions`` row is created so
858     the manager can run them. ``User.enabled`` maps to ``session.enabled``.

```

```

859     """
860     account = Account(id=user.id, telegram_id=user.id, created_at=user.created_at)
861     await upsert_account(conn, account)
862     await upsert_telegram_session(
863         conn,
864         TelegramSession(
865             account_id=user.id,
866             phone=user.phone,
867             session_path=f"user_{user.id}.session",
868             enabled=user.enabled,
869             created_at=user.created_at,
870         ),
871     )
872     return await get_user(conn, user.id) # type: ignore[return-value]
873
874
875 async def get_user(conn: aiomysqlite.Connection, user_id: int) -> User | None:
876     """Deprecated shim: project the (account, session) pair back to a ``User``."""
877     account = await get_account(conn, user_id)
878     if account is None:
879         return None
880     session = await get_telegram_session(conn, user_id)
881     return _account_session_to_user(account, session)
882
883
884 async def list_enabled_users(conn: aiomysqlite.Connection) -> list[User]:
885     """Deprecated shim: list users that have an enabled MTProto session.
886
887     Equivalent to projecting :func:`list_runnable_sessions` onto the legacy
888     ``User`` shape. Bot-only accounts (no session) are excluded ? matching the
889     pre-ADR-0010 "list enabled users" semantics where every row was runnable.
890     """
891     pairs = await list_runnable_sessions(conn)
892     return [_account_session_to_user(a, s) for a, s in pairs]
893
894
895 # --- Source repos (ADR 0011) -----
896
897
898 async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:
899     """Insert a source row, or update title/watched on conflict.
900
901     The conflict key is ``(account_id, provider, provider_source_id)`` per the
902     ADR 0011 schema. Returns the persisted row.
903     """
904     await conn.execute(
905         """INSERT INTO sources
906             (account_id, provider, provider_source_id, title, kind, watched,
907              seen_first_at, last_seen_at)
908         VALUES (?, ?, ?, ?, ?, ?, ?, ?)
909         ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
910             title=excluded.title,
911             watched=excluded.watched,
912             last_seen_at=excluded.last_seen_at""",
913         (
914             source.account_id,
915             source.provider,
916             source.provider_source_id,
917             source.title,
918             source.kind,
919             int(source.watched),
920             source.seen_first_at.isoformat(),
921             (source.last_seen_at.isoformat() if source.last_seen_at else None),
922         ),
923     )

```

```

924     await conn.commit()
925     conn.row_factory = aiosqlite.Row
926     async with conn.execute(
927         """SELECT * FROM sources
928             WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
929         (source.account_id, source.provider, source.provider_source_id),
930     ) as cur:
931         row = await cur.fetchone()
932     assert row is not None # just upserted by the conflict key
933     return _row_to_source(row)
934
935
936     async def get_source_by_provider_id(
937         conn: aiosqlite.Connection, account_id: int, provider: str, provider_source_id: str
938     ) -> Source | None:
939         """Resolve a source by its provider-native id."""
940         conn.row_factory = aiosqlite.Row
941         async with conn.execute(
942             """SELECT * FROM sources
943                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
944             (account_id, provider, provider_source_id),
945         ) as cur:
946             row = await cur.fetchone()
947         return _row_to_source(row) if row else None
948
949
950     # --- ContentItem repos (ADR 0011) -----
951
952
953     async def insert_content_item(conn: aiosqlite.Connection, item: ContentItem) ->
ContentItem:
954         """Insert a content item. Raises ``IntegrityError`` on a dedup collision.
955
956         The ``dedup_key`` must be precomputed by :func:`compute_dedup_key`. Two
957         unique constraints guard against duplicates:
958         - ``UNIQUE (account_id, source_id, provider_message_id)`` ? Tier-0.
959         - ``UNIQUE (account_id, dedup_key)`` ? Tier-1/2 (provider-neutral).
960         """
961         cur = await conn.execute(
962             """INSERT INTO content_items
963                 (account_id, source_id, provider, provider_message_id, content_type,
964                 text, link_url, media_ref, document_id, duration_s, width, height,
965                 size_bytes, mime_type, dedup_key, matched, seen_at)
966                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
967             (
968                 item.account_id,
969                 item.source_id,
970                 item.provider,
971                 item.provider_message_id,
972                 item.content_type,
973                 item.text,
974                 item.link_url,
975                 item.media_ref,
976                 item.document_id,
977                 item.duration_s,
978                 item.width,
979                 item.height,
980                 item.size_bytes,
981                 item.mime_type,
982                 item.dedup_key,
983                 int(item.matched),
984                 item.seen_at.isoformat(),
985             ),
986         )
987         await conn.commit()

```

```

988         return item.model_copy(update={"id": cur.lastrowid})
989
990
991     async def get_content_item_by_message(
992         conn: aiosqlite.Connection,
993         account_id: int,
994         source_id: int,
995         provider_message_id: str,
996     ) -> ContentItem | None:
997         """Tier-0 lookup by the (account, source, provider-message-id) natural key."""
998         conn.row_factory = aiosqlite.Row
999         async with conn.execute(
1000             """SELECT * FROM content_items
1001                WHERE account_id = ? AND source_id = ? AND provider_message_id = ?""",
1002             (account_id, source_id, provider_message_id),
1003         ) as cur:
1004             row = await cur.fetchone()
1005             return _row_to_content_item(row) if row else None
1006
1007
1008     async def get_content_item_by_dedup_key(
1009         conn: aiosqlite.Connection, account_id: int, dedup_key: str
1010     ) -> ContentItem | None:
1011         """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
1012
1013         Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014         a single query: the ``dedup_key`` already encodes whichever tier was
1015         applicable at ingestion time.
1016
1017         """
1018         conn.row_factory = aiosqlite.Row
1019         async with conn.execute(
1020             "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1021             (account_id, dedup_key),
1022         ) as cur:
1023             row = await cur.fetchone()
1024             return _row_to_content_item(row) if row else None
1025
1026 # --- Policy repos (ADR 0011, Option 2A) -----
1027
1028
1029     async def upsert_policy(conn: aiosqlite.Connection, policy: Policy) -> Policy:
1030         """Insert or update a policy row by ``(account_id, name)``.
1031
1032         Exactly one policy per account should carry ``is_default=True``; callers
1033         are responsible for flipping the previous default before flipping a new
1034         one (enforced by a partial unique index landed by future work, not in the
1035         schema ? SQLite ``CHECK`` can't express "at most one per group").
1036
1037         """
1038         await conn.execute(
1039             """INSERT INTO policies (account_id, name, config_json, is_default,
1040                                    created_at, updated_at)
1041            VALUES (?, ?, ?, ?, ?, ?)
1042            ON CONFLICT(account_id, name) DO UPDATE SET
1043                config_json=excluded.config_json,
1044                is_default=excluded.is_default,
1045                updated_at=excluded.updated_at""",
1046             (
1047                 policy.account_id,
1048                 policy.name,
1049                 policy.config_json,
1050                 int(policy.is_default),
1051                 policy.created_at.isoformat(),
1052                 policy.updated_at.isoformat(),
1053             ),
1054         )

```

```

1053     )
1054     await conn.commit()
1055     conn.row_factory = aiosqlite.Row
1056     async with conn.execute(
1057         "SELECT * FROM policies WHERE account_id = ? AND name = ?",
1058         (policy.account_id, policy.name),
1059     ) as cur:
1060         row = await cur.fetchone()
1061     assert row is not None
1062     return _row_to_policy(row)
1063
1064
1065     async def get_default_policy(conn: aiosqlite.Connection, account_id: int) -> Policy |
None:
1066         """Return the account's default policy (`is_default=1`)."""
1067         conn.row_factory = aiosqlite.Row
1068         async with conn.execute(
1069             "SELECT * FROM policies WHERE account_id = ? AND is_default = 1",
1070             (account_id,),
1071         ) as cur:
1072             row = await cur.fetchone()
1073         return _row_to_policy(row) if row else None
1074
1075
1076     # --- legacy Channel shim (backwards-compatible view over sources) -----
1077
1078
1079     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1080         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1081
1082         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1083         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1084         New code should call :func:`upsert_source` directly.
1085         """
1086         src = await upsert_source(
1087             conn,
1088             Source(
1089                 account_id=channel.user_id,
1090                 provider="telegram",
1091                 provider_source_id=str(channel.channel_id),
1092                 title=channel.title,
1093                 kind="channel",
1094                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1095                 seen_first_at=channel.seen_first_at,
1096             ),
1097         )
1098         return _source_to_channel(src)
1099
1100
1101     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1102         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1103         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1104         return _source_to_channel(src) if src else None
1105
1106
1107     def _source_to_channel(src: Source) -> Channel:
1108         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`."""
1109         return Channel(
1110             id=src.id,
1111             user_id=src.account_id,
1112             channel_id=int(src.provider_source_id),
1113             title=src.title,
1114             seen_first_at=src.seen_first_at,
1115         )

```



```

1116
1117
1118 # --- legacy VideoRecord shims (backwards-compatible view over content_items) -
1119
1120
1121 async def get_video(
1122     conn: aiosqlite.Connection, user_id: int, channel_id: int, message_id: int
1123 ) -> VideoRecord | None:
1124     """Deprecated shim: Tier-0 lookup against ``content_items`` (ADR 0011).
1125
1126     Resolves the legacy ``(user_id, channel_id, message_id)`` key to a
1127     :class:`Source` + :class:`ContentItem` and projects the latter onto a
1128     :class:`VideoRecord`. New code should call :func:`get_content_item_by_message`.
1129     """
1130     src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131     if src is None:
1132         return None
1133     item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134     return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137 async def get_video_by_document_id(
1138     conn: aiosqlite.Connection, user_id: int, document_id: int
1139 ) -> VideoRecord | None:
1140     """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141     dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143     if item is None:
1144         return None
1145     src = await get_source_by_id(conn, item.source_id)
1146     if src is None:
1147         return None
1148     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1149
1150
1151 async def get_video_by_content_fingerprint(
1152     conn: aiosqlite.Connection,
1153     user_id: int,
1154     size_bytes: int,
1155     duration_s: int,
1156     height: int,
1157     width: int,
1158 ) -> VideoRecord | None:
1159     """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
1160     dedup_key = compute_dedup_key(
1161         provider="telegram",
1162         size_bytes=size_bytes,
1163         duration_s=duration_s,
1164         height=height,
1165         width=width,
1166     )
1167     item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1168     if item is None:
1169         return None
1170     src = await get_source_by_id(conn, item.source_id)
1171     if src is None:
1172         return None
1173     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1174
1175
1176 async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
1177     """Resolve a source by its surrogate PK (used by the legacy video shims)."""
1178     conn.row_factory = aiosqlite.Row
1179     async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
1180         row = await cur.fetchone()

```

```

1181         return _row_to_source(row) if row else None
1182
1183
1184 async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
1185     """Deprecated shim: insert into ``content_items`` (ADR 0011).
1186
1187     Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
1188     if absent), computes the provider-neutral ``dedup_key`` via
1189     :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
1190     ``VideoRecord`` shape is reconstructed from the inserted row.
1191     """
1192     src = await upsert_source(
1193         conn,
1194         Source(
1195             account_id=video.user_id,
1196             provider="telegram",
1197             provider_source_id=str(video.channel_id),
1198             kind="channel",
1199             watched=True,
1200         ),
1201     )
1202     assert src.id is not None # upsert_source populates the surrogate PK
1203     item = ContentItem(
1204         account_id=video.user_id,
1205         source_id=src.id,
1206         provider="telegram",
1207         provider_message_id=str(video.message_id),
1208         content_type="video",
1209         document_id=str(video.document_id) if video.document_id is not None else None,
1210         duration_s=video.duration_s,
1211         width=video.width,
1212         height=video.height,
1213         size_bytes=video.size_bytes,
1214         mime_type=video.mime_type,
1215         dedup_key=compute_dedup_key(
1216             provider="telegram",
1217             document_id=(str(video.document_id) if video.document_id is not None else
1218 None),
1219             source_id=src.id,
1220             provider_message_id=str(video.message_id),
1221             size_bytes=video.size_bytes,
1222             duration_s=video.duration_s,
1223             height=video.height,
1224             width=video.width,
1225         ),
1226         matched=video.matched,
1227         seen_at=video.seen_at,
1228     )
1229     item = await insert_content_item(conn, item)
1230     return _content_item_to_video(item, src, video.user_id, video.channel_id)
1231
1232
1233 async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
1234     """Deprecated shim: no-op for the legacy ``forwarded`` flag (ADR 0011).
1235
1236     Per-destination delivery state moves to ``delivery_attempts`` (Track C,
1237     #28). For Phase 1 (before Track C lands) there is no equivalent column on
1238     ``content_items`` ? the ``matched`` flag alone is sufficient because v1
1239     delivery still targets the single ``delivery_target`` and retried delivery
1240     is harmless (forwards are idempotent). This shim is retained so existing
1241     call sites in ``delivery.py`` keep working without a flag day.
1242     """
1243     # content_items has no ``forwarded`` column post-ADR-0011; nothing to write.
1244     # The row's ``matched`` flag (set at insert time) is the surviving signal.
1245     return None

```

```

1245
1246
1247 def _content_item_to_video(
1248     item: ContentItem, src: Source, user_id: int, channel_id: int
1249 ) -> VideoRecord:
1250     """Project a :class:`ContentItem` onto the legacy :class:`VideoRecord`.
1251
1252     ``forwarded`` is always ``False`` post-ADR-0011 (the column is dropped;
1253     per-destination delivery state moves to Track C). Existing callers read it
1254     only to decide whether to skip re-delivery; since forwards are idempotent
1255     the ``False`` default is safe for Phase 1.
1256     """
1257     return VideoRecord(
1258         id=item.id,
1259         user_id=user_id,
1260         channel_id=channel_id,
1261         message_id=int(item.provider_message_id),
1262         document_id=int(item.document_id) if item.document_id is not None else None,
1263         duration_s=item.duration_s,
1264         width=item.width,
1265         height=item.height,
1266         size_bytes=item.size_bytes,
1267         mime_type=item.mime_type,
1268         matched=item.matched,
1269         forwarded=False,
1270         seen_at=item.seen_at,
1271     )
1272
1273
1274 # --- filter config shims (backwards-compatible view over policies) -----
1275
1276
1277 async def load_filter_config(conn: aiomysqlite.Connection, user_id: int) -> FilterConfig:
1278     """Deprecated shim: load the account's default :class:`Policy` as FilterConfig.
1279
1280     ADR 0011 (Option 2A): ``filter_config`` is renamed to
1281     ``filter_config_deprecated``; this reads ``policies WHERE is_default=1``
1282     instead. Falls back to :data:`FilterConfig` defaults if no policy exists.
1283     """
1284     policy = await get_default_policy(conn, user_id)
1285     if policy is None:
1286         return FilterConfig()
1287     return FilterConfig.model_validate_json(policy.config_json)
1288
1289
1290 async def save_filter_config(conn: aiomysqlite.Connection, user_id: int, cfg:
FilterConfig) -> None:
1291     """Deprecated shim: upsert the account's default :class:`Policy`."""
1292     existing = await get_default_policy(conn, user_id)
1293     await upsert_policy(
1294         conn,
1295         Policy(
1296             id=existing.id if existing is not None else None,
1297             account_id=user_id,
1298             name="default",
1299             config_json=cfg.model_dump_json(),
1300             is_default=True,
1301             created_at=(existing.created_at if existing is not None else _now()),
1302             updated_at=_now(),
1303         ),
1304     )
1305
1306
1307 # --- bulk helpers (used by the manager on startup) -----
1308

```

```

1309
1310 async def list_videos_for_user( # v2: used by on-demand /search; not called in v1
1311     conn: aiosqlite.Connection, user_id: int, limit: int = 100
1312 ) -> list[VideoRecord]:
1313     """Deprecated shim: list the account's content items projected onto VideoRecord."""
1314     conn.row_factory = aiosqlite.Row
1315     async with conn.execute(
1316         """SELECT ci.*, s.provider_source_id AS src_pid
1317             FROM content_items ci
1318             JOIN sources s ON s.id = ci.source_id
1319             WHERE ci.account_id = ?
1320             ORDER BY ci.id DESC LIMIT ?""",
1321         (user_id, limit),
1322     ) as cur:
1323         rows = await cur.fetchall()
1324     return [
1325         VideoRecord(
1326             id=r["id"],
1327             user_id=user_id,
1328             channel_id=int(r["src_pid"]),
1329             message_id=int(r["provider_message_id"]),
1330             document_id=(int(r["document_id"]) if r["document_id"] is not None else
1331 None),
1332             duration_s=r["duration_s"],
1333             width=r["width"],
1334             height=r["height"],
1335             size_bytes=r["size_bytes"],
1336             mime_type=r["mime_type"],
1337             matched=bool(r["matched"]),
1338             forwarded=False,
1339             seen_at=r["seen_at"],
1340         )
1341         for r in rows
1342     ]
1343
1344 __all__ = [
1345     "init_db",
1346     "compute_dedup_key",
1347     # Account + session repos (ADR 0010)
1348     "upsert_account",
1349     "get_account",
1350     "get_account_by_telegram_id",
1351     "upsert_telegram_session",
1352     "get_telegram_session",
1353     "list_runnable_sessions",
1354     # Provider-neutral domain repos (ADR 0011)
1355     "upsert_source",
1356     "get_source_by_provider_id",
1357     "get_source_by_id",
1358     "insert_content_item",
1359     "get_content_item_by_message",
1360     "get_content_item_by_dedup_key",
1361     "upsert_policy",
1362     "get_default_policy",
1363     # Legacy User shims (backwards-compatible)
1364     "upsert_user",
1365     "get_user",
1366     "list_enabled_users",
1367     # Legacy Channel / Video / config shims (ADR 0011)
1368     "upsert_channel",
1369     "get_channel",
1370     "get_video",
1371     "get_video_by_document_id",
1372     "get_video_by_content_fingerprint",

```

```

1373         "insert_video",
1374         "mark_forwarded",
1375         "load_filter_config",
1376         "save_filter_config",
1377         "list_videos_for_user",
1378     ]
1379
1380
1381     def __dir__() -> Iterable[str]:
1382         return __all__
1383

```

5. assistant (2026-07-06T06:03:00.698Z)

Now let me look at delivery.py to trace the `mark\_forwarded` / `forwarded` state dependency, which is the core question about double-delivery.

6. user (2026-07-06T06:03:01.344Z)

src\tg_video_filter\delivery.py

7. user (2026-07-06T06:03:02.352Z)

```

src\tg_video_filter\delivery.py:10: ``seconds + jitter``, retry once) and mark the DB row
`forwarded=1` on success.
src\tg_video_filter\delivery.py:138:         await db.mark_forwarded(conn, video_id)
src\tg_video_filter\delivery.py:139:         log.info("video_forwarded video=%s target=%s",
video_id, target)
src\tg_video_filter\delivery.py:156:         await db.mark_forwarded(conn, video_id)
src\tg_video_filter\backfill.py:165:             existing = await db.get_video(conn, user.id,
channel_id, message_id)
src\tg_video_filter\backfill.py:169:             # Tier 1: same Telegram document.id
(forwarded / cross-posted file).
src\tg_video_filter\backfill.py:171:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\backfill.py:192:             existing = await
db.get_video_by_content_fingerprint(
src\tg_video_filter\backfill.py:230:             matched=result.matched,
src\tg_video_filter\backfill.py:234:             if result.matched:
src\tg_video_filter\telethon_client.py:7:         -> dedup via db.get_video()
src\tg_video_filter\telethon_client.py:187:             existing = await db.get_video(conn, user.id,
channel_id, message_id)
src\tg_video_filter\telethon_client.py:191:             # Tier 1: same Telegram document.id ?
forwarded/cross-posted file.
src\tg_video_filter\telethon_client.py:193:             existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\telethon_client.py:214:             existing = await
db.get_video_by_content_fingerprint(
src\tg_video_filter\telethon_client.py:260:             matched=result.matched,
src\tg_video_filter\telethon_client.py:283:             result.matched,
src\tg_video_filter\telethon_client.py:287:             if result.matched:
src\tg_video_filter\telethon_client.py:320:             same ``document.id`` when a file is forwarded
across channels.
src\tg_video_filter\models.py:322:         forwarded: bool = False
src\tg_video_filter\db.py:72:         forwarded INTEGER NOT NULL DEFAULT 0,
src\tg_video_filter\db.py:455:         forwarded INTEGER NOT NULL DEFAULT 0,
src\tg_video_filter\db.py:461:         "height, size_bytes, mime_type, matched, forwarded,
seen_at",
src\tg_video_filter\db.py:560:         v.matched,
src\tg_video_filter\db.py:673:         forwarded=bool(row["forwarded"]),
src\tg_video_filter\db.py:983:         int(item.matched),
src\tg_video_filter\db.py:1121:async def get_video(
src\tg_video_filter\db.py:1137:async def get_video_by_document_id(
src\tg_video_filter\db.py:1151:async def get_video_by_content_fingerprint(

```

```

src\tg_video_filter\db.py:1225:         matched=video.matched,
src\tg_video_filter\db.py:1232:async def mark_forwarded(conn: aiosqlite.Connection, video_id:
int) -> None:
src\tg_video_filter\db.py:1233:     """Deprecated shim: no-op for the legacy ``forwarded`` flag
(ADR 0011).
src\tg_video_filter\db.py:1242:     # content_items has no ``forwarded`` column post-ADR-0011;
nothing to write.
src\tg_video_filter\db.py:1252:     ``forwarded`` is always ``False`` post-ADR-0011 (the column
is dropped;
src\tg_video_filter\db.py:1268:         matched=item.matched,
src\tg_video_filter\db.py:1269:         forwarded=False,
src\tg_video_filter\db.py:1337:         forwarded=False,
src\tg_video_filter\db.py:1370:     "get_video",
src\tg_video_filter\db.py:1371:     "get_video_by_document_id",
src\tg_video_filter\db.py:1372:     "get_video_by_content_fingerprint",
src\tg_video_filter\db.py:1374:     "mark_forwarded",
src\tg_video_filter\filters\__init__.py:12:     if result.matched:
src\tg_video_filter\filters\__init__.py:50:         if not r.matched:

```

8. assistant (2026-07-06T06:03:05.094Z)

Now let me trace how `forwarded` state is used to avoid double-delivery. Let me read the relevant parts of delivery.py, telethon_client.py, and backfill.py.

9. user (2026-07-06T06:03:05.791Z)

```

1     """Delivery: send matched videos to the user's Saved Messages.
2
3     Three delivery modes (per-user, configured via FilterConfig.delivery_mode):
4
5     - ``FORWARD``: forward the video itself (re-uploads to Saved Messages).
6     - ``LINK``: send a text message with a t.me deep link + metadata summary.
7     - ``BOTH``: forward the video AND send the link message.
8
9     All modes wrap their Telethon calls with ``FloodWaitError`` handling (sleep
10     ``seconds + jitter``, retry once) and mark the DB row ``forwarded=1`` on success.
11     """
12
13     from __future__ import annotations
14
15     import asyncio
16     import logging
17     import random
18     from collections.abc import Awaitable, Callable
19
20     import aiosqlite
21     from telethon import TelegramClient
22     from telethon.errors import FloodWaitError
23     from telethon.tl.types import Message
24
25     from tg_video_filter import db
26     from tg_video_filter.models import DeliveryMode
27
28     log = logging.getLogger(__name__)
29
30     # Extra jitter (seconds) added on top of the requested flood wait so we don't
31     # retry at the exact boundary and risk re-triggering the limit.
32     _FLOOD_JITTER_RANGE = (1.0, 5.0)
33
34
35     # --- pure link-message formatting -----
36
37
38     def _format_duration(seconds: int | None) -> str:
39         """Human-readable duration for the link summary."""

```

```

40         if seconds is None:
41             return "?"
42         if seconds < 60:
43             return f"{seconds}s"
44         if seconds < 3600:
45             return f"{seconds // 60}min"
46         h = seconds // 3600
47         m = (seconds % 3600) // 60
48         return f"{h}h{m:02d}min" if m else f"{h}h"
49
50
51     def _format_size(size_bytes: int | None) -> str:
52         """Human-readable file size for the link summary."""
53         if size_bytes is None:
54             return "?"
55         mib = size_bytes / (1024 * 1024)
56         if mib >= 1024:
57             return f"{mib / 1024:.1f}GB"
58         return f"{mib:.0f}MB"
59
60
61     def _format_height(height: int | None) -> str:
62         """Resolution label: 1080 -> 1080p, None -> ?."""
63         if height is None:
64             return "?"
65         return f"{height}p"
66
67
68     def build_link_text(
69         *,
70         message_link: str | None,
71         duration_s: int | None,
72         height: int | None,
73         size_bytes: int | None,
74         channel_title: str | None,
75     ) -> str:
76         """Build the text message sent in LINK/BOTH mode.
77
78         Format::
79
80             ? 1080p · 5min · 480MB
81             #ChannelName
82             https://t.me/c/1234567890/42
83
84         ``message_link`` may be None if the link can't be constructed (e.g. the
85         channel is inaccessible); in that case the link line is omitted.
86         """
87         lines = [
88             f"? {_format_height(height)} · {_format_duration(duration_s)} · "
89             f"{_format_size(size_bytes)}"
90         ]
91         if channel_title:
92             # Sanitize: replace spaces in the title for a hashtag-like display.
93             lines.append(f"##{channel_title.replace(' ', '_')}")
94         if message_link:
95             lines.append(message_link)
96         return "\n".join(lines)
97
98     # --- Telethon interaction (retry/flood logic) -----
99
100
101     async def _send_with_retry(
102         client: TelegramClient,
103         send_callable: Callable[[], Awaitable[None]],

```

```

104         video_id: int,
105     ) -> bool:
106         """Execute a send/forward callable with FloodWait handling. Returns success."""
107         try:
108             await send_callable()
109         except FloodWaitError as e:
110             wait = e.seconds + random.uniform(*_FLOOD_JITTER_RANGE)
111             log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id,
e.seconds, wait)
112             await asyncio.sleep(wait)
113             try:
114                 await send_callable()
115             except Exception:
116                 log.exception("delivery_retry_failed video=%s", video_id)
117                 return False
118         except Exception:
119             log.exception("delivery_failed video=%s", video_id)
120             return False
121         return True
122
123
124     async def forward_to_target(
125         client: TelegramClient,
126         message: Message,
127         conn: aiosqlite.Connection,
128         video_id: int,
129         target: str = "me",
130     ) -> bool:
131         """Forward ``message`` to ``target``, marking the DB row on success.
132
133         ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"`` ,
134         or a numeric chat id string. See ADR 0005.
135         """
136         ok = await _send_with_retry(client, lambda: client.forward_messages(target,
message), video_id)
137         if ok:
138             await db.mark_forwarded(conn, video_id)
139             log.info("video_forwarded video=%s target=%s", video_id, target)
140         return ok
141
142
143     async def send_link_message(
144         client: TelegramClient,
145         text: str,
146         conn: aiosqlite.Connection,
147         video_id: int,
148         target: str = "me",
149     ) -> bool:
150         """Send a link/summary text message to ``target``, marking the DB row.
151
152         ``target`` is a Telethon entity; see :func:`forward_to_target`.
153         """
154         ok = await _send_with_retry(client, lambda: client.send_message(target, text),
video_id)
155         if ok:
156             await db.mark_forwarded(conn, video_id)
157             log.info("link_sent video=%s target=%s", video_id, target)
158         return ok
159
160
161     async def deliver(
162         client: TelegramClient,
163         message: Message,
164         conn: aiosqlite.Connection,
165         video_id: int,

```



```

166         mode: DeliveryMode,
167         *,
168         link_text: str | None = None,
169         target: str = "me",
170     ) -> bool:
171         """Dispatch delivery based on ``mode`` to ``target``.
172
173         ``link_text`` is required for LINK/BOTH modes (built by the caller via
174         :func:`build_link_text`). If it's None in LINK/BOTH mode, falls back to
175         FORWARD so we never silently drop a matched video.
176
177         ``target`` is a Telethon entity (``"me"`` , ``"@username"`` , chat id).
178         See ADR 0005.
179         """
180         if mode == DeliveryMode.LINK and link_text is not None:
181             return await send_link_message(client, link_text, conn, video_id, target)
182         if mode == DeliveryMode.BOTH and link_text is not None:
183             fwd_ok = await forward_to_target(client, message, conn, video_id, target)
184             link_ok = await send_link_message(client, link_text, conn, video_id, target)
185             if not link_ok:
186                 log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
187             if not fwd_ok:
188                 log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
189             return fwd_ok or link_ok
190         # FORWARD, or fallback when link_text is missing.
191         if mode != DeliveryMode.FORWARD:
192             log.warning(
193                 "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
194                 video_id,
195                 mode.value,
196                 link_text is None,
197             )
198         return await forward_to_target(client, message, conn, video_id, target)
199
200
201     __all__ = [
202         "build_link_text",
203         "deliver",
204         "forward_to_target",
205         "send_link_message",
206     ]
207

```

10. assistant (2026-07-06T06:03:10.272Z)

Now let me trace how the caller (telethon_client.py) decides whether to deliver, and whether `forwarded` is consulted to skip re-delivery across restarts.

11. user (2026-07-06T06:03:11.406Z)

```

175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- three-tier dedup (cheapest first) ---
185
186         # Tier 0: same (user, channel, message) instance ? already processed.
187         existing = await db.get_video(conn, user.id, channel_id, message_id)
188         if existing is not None:
189             return

```

```

190
191 # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
192 if document_id is not None:
193     existing = await db.get_video_by_document_id(conn, user.id, document_id)
194     if existing is not None:
195         log.info(
196             "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
197             "original_channel=%s original_msg=%s",
198             user.id,
199             document_id,
200             channel_id,
201             message_id,
202             existing.channel_id,
203             existing.message_id,
204         )
205         return
206
207 # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208 if (
209     meta.size_bytes is not None
210     and meta.duration_s is not None
211     and meta.height is not None
212     and meta.width is not None
213 ):
214     existing = await db.get_video_by_content_fingerprint(
215         conn,
216         user.id,
217         meta.size_bytes,
218         meta.duration_s,
219         meta.height,
220         meta.width,
221     )
222     if existing is not None:
223         log.info(
224             "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225             "channel=%s msg=%s original_channel=%s original_msg=%s",
226             user.id,
227             meta.size_bytes,
228             meta.duration_s,
229             meta.height,
230             meta.width,
231             channel_id,
232             message_id,
233             existing.channel_id,
234             existing.message_id,
235         )
236         return
237
238 # Lazily record the channel (best-effort; failures are non-fatal).
239 try:
240     await db.upsert_channel(
241         conn,
242         Channel(user_id=user.id, channel_id=channel_id,
243             title=_channel_title(event)),
244     )
245 except Exception:
246     log.warning(
247         "failed to upsert channel %s for user %s", channel_id, user.id,
248     )
249 exc_info=True
250
251 result = engine_holder[ENGINE_KEY].evaluate(meta)
252 record = VideoRecord(
253     user_id=user.id,
254     channel_id=channel_id,

```

```

253         message_id=message_id,
254         document_id=document_id,
255         duration_s=meta.duration_s,
256         width=meta.width,
257         height=meta.height,
258         size_bytes=meta.size_bytes,
259         mime_type=meta.mime_type,
260         matched=result.matched,
261     )
262     try:
263         inserted = await db.insert_video(conn, record)
264     except sqlite3.IntegrityError:
265         # Lost a dedup race: a concurrent delivery of the same file
266         # (same document_id or same message instance) won the insert.
267         # The three-tier check above already classified this as new, so
268         # the collision means another task is processing (or has
269         # processed) it ? log and skip.
270         log.info(
271             "video_dedup_race user=%s channel=%s msg=%s doc_id=%s",
272             user.id,
273             channel_id,
274             message_id,
275             document_id,
276         )
277         return
278     log.info(
279         "video_seen user=%s channel=%s msg=%s matched=%s reasons=%s",
280         user.id,
281         channel_id,
282         message_id,
283         result.matched,
284         result.reasons,
285     )
286
287     if result.matched:
288         # Build link text for LINK/BOTH modes (None for FORWARD).
289         link_text = None
290         if delivery_mode != DeliveryMode.FORWARD:
291             channel_title = _channel_title(event)
292             link_text = build_link_text(
293                 message_link=getattr(message, "link", None),
294                 duration_s=meta.duration_s,
295                 height=meta.height,
296                 size_bytes=meta.size_bytes,
297                 channel_title=channel_title,
298             )
299         await deliver(
300             client,
301             message,
302             conn,
303             inserted.id,
304             delivery_mode,

```

12. assistant (2026-07-06T06:03:14.844Z)

Let me see the rest of the live-path handler and understand whether delivery here relies on `forwarded` for restart safety. The live path delivers immediately after insert (in the same event). Let me check backfill.py, which runs on startup ? that's the restart re-forward risk.

13. user (2026-07-06T06:03:15.737Z)

```

150         if msg_date is not None and msg_date < cutoff:
151             break
152
153         dialog_total += 1

```

```

154     meta = extract_video_meta(message) # type: ignore[arg-type]
155     if meta is None:
156         continue
157
158     channel_id = _peer_id(getattr(message, "peer_id", None) or 0)
159     message_id = message.id
160     document_id = _extract_document_id(message) # type: ignore[arg-type]
161
162     # --- three-tier dedup (cheapest first) ---
163
164     # Tier 0: same (user, channel, message) instance.
165     existing = await db.get_video(conn, user.id, channel_id, message_id)
166     if existing is not None:
167         continue
168
169     # Tier 1: same Telegram document.id (forwarded / cross-posted file).
170     if document_id is not None:
171         existing = await db.get_video_by_document_id(conn, user.id,
172 document_id)
173         if existing is not None:
174             log.info(
175                 "backfill_dedup_document_id user=%s doc_id=%s channel=%s
176 msg=%s "
177                 "original_channel=%s original_msg=%s",
178                 user.id,
179                 document_id,
180                 channel_id,
181                 message_id,
182                 existing.channel_id,
183                 existing.message_id,
184             )
185             continue
186
187     # Tier 2: metadata fingerprint (re-uploaded identical file).
188     if (
189         meta.size_bytes is not None
190         and meta.duration_s is not None
191         and meta.height is not None
192         and meta.width is not None
193     ):
194         existing = await db.get_video_by_content_fingerprint(
195             conn,
196             user.id,
197             meta.size_bytes,
198             meta.duration_s,
199             meta.height,
200             meta.width,
201         )
202         if existing is not None:
203             log.info(
204                 "backfill_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s
205 "
206                 "channel=%s msg=%s original_channel=%s original_msg=%s",
207                 user.id,
208                 meta.size_bytes,
209                 meta.duration_s,
210                 meta.height,
211                 meta.width,
212                 channel_id,
213                 message_id,
214                 existing.channel_id,
215                 existing.message_id,
216             )
217             continue

```

```

216         summary["videos_scanned"] += 1
217         dialog_videos += 1
218
219         result = engine.evaluate(meta)
220         record = VideoRecord(
221             user_id=user.id,
222             channel_id=channel_id,
223             message_id=message_id,
224             document_id=document_id, # type: ignore[arg-type]
225             duration_s=meta.duration_s,
226             width=meta.width,
227             height=meta.height,
228             size_bytes=meta.size_bytes,
229             mime_type=meta.mime_type,
230             matched=result.matched,
231         )
232         inserted = await db.insert_video(conn, record)
233
234         if result.matched:
235             summary["videos_matched"] += 1
236             dialog_matched += 1
237             link_text = build_link_text(
238                 message_link=getattr(message, "link", None),
239                 duration_s=meta.duration_s,
240                 height=meta.height,
241                 size_bytes=meta.size_bytes,
242                 channel_title=title, # from the dialog object
243             )
244             await deliver(
245                 client,
246                 message,
247                 conn,
248                 inserted.id or 0,
249                 delivery_mode,
250                 link_text=link_text,
251                 target=delivery_target,
252             )
253
254             # Periodic progress (at most once per 30 seconds per dialog).
255             now = asyncio.get_event_loop().time()
256             if progress_callback and now - last_progress > 30:
257                 await progress_callback(
258                     f"    #{title}: {dialog_total} msgs, {dialog_videos} videos,
{dialog_matched} matched..."
259                 )

```

14. assistant (2026-07-06T06:03:22.068Z)

Now I understand the delivery model. Both the live path and backfill ****gate** delivery on Tier-0 dedup (``get_video``) returning `None` ? i.e. the item not yet being in the DB. Delivery happens once, immediately after insert, in the same process run. The ``forwarded`` flag is NOT consulted anywhere to decide whether to deliver. Let me confirm this by checking whether ``forwarded`` field is ever read to skip delivery, and check v1 behavior on main.

15. user (2026-07-06T06:03:23.140Z)

No matches found

16. user (2026-07-06T06:03:24.602Z)

```

66:     forwarded    INTEGER NOT NULL DEFAULT 0,
286:             forwarded    INTEGER NOT NULL DEFAULT 0,
292:             "height, size_bytes, mime_type, matched, forwarded, seen_at",
406:             forwarded=bool(row["forwarded"]),

```

```

627:     Telegram reuses ``document.id`` when the same file is forwarded across
676:         same file forwarded across channels (content dedup Tier 1).
681:         size_bytes, mime_type, matched, forwarded, seen_at)
694:         int(video.forwarded),
702:async def mark_forwarded(conn: aiosqlite.Connection, video_id: int) -> None:
704:     await conn.execute("UPDATE videos SET forwarded = 1 WHERE id = ?", (video_id,))
768:     "mark_forwarded",

```

17. assistant (2026-07-06T06:03:33.298Z)

Confirmed: `forwarded` is never read to gate delivery, on either branch or main. On main, `mark\_forwarded` did an `UPDATE videos SET forwarded = 1`, but no code ever reads that flag to skip re-delivery. Delivery is gated purely by `get\_video` (Tier-0 presence). Since `insert\_video` runs *before* `deliver`, and `matched` is stored at insert, a restart re-scan finds the item already present (Tier-0) and skips it entirely ? so it never reaches delivery. So `mark\_forwarded` being a no-op does not introduce a new double-delivery bug: the behavior is identical to main (the flag was already dead for delivery gating). This is not a defect in this PR.

Now let me focus on the `int(...)` cast paths ? the more promising area. Let me examine `\_content\_item\_to\_video`, `get\_video\_by\_document\_id`, and `list\_videos\_for\_user` for non-numeric/None cast failures. The key question: can `int(item.document\_id)` or `int(src.provider\_source\_id)` raise?

Let me look at `get\_video\_by\_document\_id` (line 1148) carefully. It calls `\_content\_item\_to\_video(item, src, user\_id, int(src.provider\_source\_id))`. If `provider\_source\_id` is non-numeric ? ValueError. Let me check the tests and think about who writes content_items.

18. user (2026-07-06T06:03:33.907Z)

```

1      """Tests for the SQLite data layer (db.py) using an in-memory database."""
2
3      from __future__ import annotations
4
5      import sqlite3
6
7      import aiosqlite
8      import pytest
9
10     from tg_video_filter import db
11     from tg_video_filter.models import (
12         DEFAULT_FILTER_CONFIG,
13         Account,
14         Channel,
15         ContentItem,
16         FilterConfig,
17         Policy,
18         Source,
19         TelegramSession,
20         User,
21         VideoRecord,
22     )
23
24     USER_ID = 111
25     CHANNEL_ID = -1001234567890
26
27
28     @pytest.fixture
29     async def conn() -> aiosqlite.Connection:
30         """An in-memory SQLite connection with the schema applied."""
31         c = await aiosqlite.connect(":memory:")
32         await db._apply_schema(c)
33         yield c
34         await c.close()

```

```

35
36
37     async def _seed_user(conn: aiosqlite.Connection, uid: int = USER_ID) -> User:
38         return await db.upsert_user(conn, User(id=uid, phone="+15555550100"))
39
40
41     # --- schema / init -----
42
43
44     async def test_init_db_creates_tables(tmp_path: object) -> None:
45         db_path = str(tmp_path) + "/test.db" # type: ignore[operator]
46         await db.init_db(db_path)
47         # Re-open and confirm all expected tables exist. Post-ADR-0011 the
48         # provider-neutral tables are sources/content_items/policies (plus the
49         # empty Track-C tables destinations/routing_rules/delivery_attempts);
50         # the legacy channels/videos/filter_config are absent on fresh DBs
51         # (they only exist as *_deprecated on migrated legacy DBs).
52         async with (
53             aiosqlite.connect(db_path) as c,
54             c.execute("SELECT name FROM sqlite_master WHERE type='table' ORDER BY name") as
cur,
55         ):
56             names = [r[0] for r in await cur.fetchall()]
57             assert set(names) >= {
58                 "accounts",
59                 "telegram_sessions",
60                 "sources",

```

19. assistant (2026-07-06T06:03:44.265Z)

Now let me think through the ``int(...)`` cast risk concretely, and consider the ``get_video_by_document_id`` path specifically. The concern from the prompt: a `content_item` created by non-shim code (e.g. Track C, or a future provider, or even the migration-003 backfill). But scope is THIS PR's diff. Within this PR, who can create `content_items` with a non-numeric ``provider_message_id``, ``document_id``, or a source with non-numeric ``provider_source_id``?

The important realistic case: `**`get_video_by_document_id`` calls ``int(src.provider_source_id)``. All the shim writers use numeric telegram ids, so within the shim's own usage this is fine. But there's a subtler bug. Let me re-examine ``get_video_by_document_id`` for a `**cross-source dedup_key collision**` issue and the ``int()`` on `document_id`.

Let me look carefully at a real correctness issue: in ``_content_item_to_video``, ``message_id=int(item.provider_message_id)``. And ``list_videos_for_user`` does ``channel_id=int(r["src_pid"])``. These will raise if the underlying value is non-numeric. Let me check whether the schema/insert path allows a text `provider_source_id` to reach a shim reader. Let me write a targeted repro to test the most likely real bug: ``get_video_by_document_id`` when the document is in a source whose `provider_source_id` is... always numeric for telegram.

Let me instead reconsider ``mark_forwarded`` more carefully for the `**BOTH mode**` case, and reconsider whether there's a genuine regression. Let me first check the `test_delivery.py` diff to understand what behavior the PR author changed there.

20. user (2026-07-06T06:03:45.445Z)

```

diff --git a/tests/test_delivery.py b/tests/test_delivery.py
index 61d42fa..e74206e 100644
--- a/tests/test_delivery.py
+++ b/tests/test_delivery.py
@@ -227,15 +227,25 @@ async def test_deliver_flood_then_retry(
     assert client.forward_calls == 2 # initial + 1 retry

-async def test_deliver_marks_forwarded_in_db(
+async def test_deliver_succeeds_without_forwarded_flag_post_adr_0011(

```

```

        db_with_video: tuple[aiosqlite.Connection, int],
    ) -> None:
+     """ADR 0011: delivery succeeds but ``forwarded`` stays False (no-op shim).
+
+     Post-migration-003 there is no ``forwarded`` column on ``content_items``;
+     per-destination delivery state moves to ``delivery_attempts`` in Track C.
+     For Phase 1 ``mark_forwarded`` is a no-op, so the projected
+     ``VideoRecord.forwarded`` is always False ? but delivery still returns True.
+     """
+     c, vid = db_with_video
+     client = FakeClient()
-     await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text="test") # type:
ignore[arg-type]
+     ok = await deliver(client, _SENTINEL_MSG, c, vid, DeliveryMode.LINK, link_text="test") #
type: ignore[arg-type]
+     assert ok is True
+     row = await db.get_video(c, USER_ID, CHANNEL_ID, MSG_ID)
+     assert row is not None
-     assert row.forwarded is True
+     # forwarded is always False post-ADR-0011 (the column is dropped; delivery
+     # state moves to Track C). The match is still recorded via `matched`.
+     assert row.forwarded is False

```

```

async def test_both_mode_logs_warning_on_link_failure(

```

21. assistant (2026-07-06T06:03:59.043Z)

This confirms the author consciously changed the test to assert `forwarded is False` and delivery still works. This is a documented behavioral change, not a regression, since nothing gates on `forwarded`.

Now let me dig into the real cast-failure risk. Let me examine `get\_video\_by\_document\_id` (line 1148) more closely. It computes `dedup\_key = "tg:doc:{document\_id}"`, looks up by dedup_key, then `int(src.provider\_source\_id)`. Consider the **dedup_key collision across accounts** ? no, it's scoped by account_id.

The real concern: the `dedup\_key` uniqueness is `UNIQUE(account\_id, dedup\_key)`. But a `tg:doc:` dedup_key could match a content_item whose `document\_id` column is NULL but whose dedup_key happens to be `tg:doc:X` ? no, that's not possible via the shim. Let me test the actual `int()` risk with a repro against the fingerprint path, which is the interesting one. In `get\_video\_by\_content\_fingerprint`, the returned item may have `document\_id=None` and `\_content\_item\_to\_video` handles None. But `int(src.provider\_source\_id)` ? provider_source_id is always numeric for telegram sources.

Let me look for the genuine bug. Let me re-read `get\_video\_by\_document\_id` vs how document_id is stored. In `insert\_video` line 1209, `document\_id=str(video.document\_id)`. And dedup_key uses `document\_id` (str). Fine. Let me now test the actual scenario the prompt flags: **a content_item created by non-shim code with non-numeric provider_message_id**. Within THIS diff, migration 003 backfills content_items from `videos` ? always numeric. So no in-diff writer produces non-numeric values.

But wait ? there IS an in-diff path worth checking: `insert\_content\_item` is public API in this PR (exported in `\_\_all\_\_`), and `list\_videos\_for\_user` / the video shims read ALL content_items for an account regardless of who wrote them. Let me verify: is `list\_videos\_for\_user` reachable over content_items that a legitimately provider-neutral writer could create with a non-numeric provider_message_id? That's Track C/future, out of this diff's writers. Let me confirm no other caller exists.

22. user (2026-07-06T06:04:00.311Z)

```

src\tg_video_filter\db.py:953:async def insert_content_item(conn: aiosqlite.Connection, item:
ContentItem) -> ContentItem:
src\tg_video_filter\db.py:1137:async def get_video_by_document_id(
src\tg_video_filter\db.py:1151:async def get_video_by_content_fingerprint(

```



```

src\tg_video_filter\db.py:1228: item = await insert_content_item(conn, item)
src\tg_video_filter\db.py:1310:async def list_videos_for_user( # v2: used by on-demand /search;
not called in v1
src\tg_video_filter\db.py:1358: "insert_content_item",
src\tg_video_filter\db.py:1371: "get_video_by_document_id",
src\tg_video_filter\db.py:1372: "get_video_by_content_fingerprint",
src\tg_video_filter\db.py:1377: "list_videos_for_user",
src\tg_video_filter\backfill.py:171: existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\backfill.py:192: existing = await
db.get_video_by_content_fingerprint(
src\tg_video_filter\telethon_client.py:193: existing = await
db.get_video_by_document_id(conn, user.id, document_id)
src\tg_video_filter\telethon_client.py:214: existing = await
db.get_video_by_content_fingerprint(

```

23. assistant (2026-07-06T06:04:08.168Z)

Now the critical realization about ``get_video_by_document_id``. Let me trace a REAL bug scenario. The dedup lookup by ``tg:doc:{document_id}`` is scoped only by ``account_id``, NOT by source. So it can return a content_item from a ****different source**** (different channel) than the one being scanned ? that's the whole point of Tier-1 cross-channel dedup. The function then computes ``channel_id=int(src.provider_source_id)`` where ``src`` is the **original** item's source. That's correct behavior (returns the original's channel).

But here's the genuine concern: ****`int(src.provider\_source\_id)`****. For a ``content_item`` whose source is a Telegram channel, ``provider_source_id`` is a stringified numeric channel_id ? `int()` works. So no crash in the Telegram-only world.

Let me now focus and verify the strongest candidate finding with a concrete repro. Let me test whether ``get_video_by_document_id`` returns wrong ``channel_id`/`message_id`` when the document was first seen in a DIFFERENT channel. This matters because the caller logs ``existing.channel_id`` and ``existing.message_id``. Let me also test the ``document_id`` collision between the ``tg:doc:`` and ``tg:fp:`` cases. Let me write a focused repro.

24. assistant (2026-07-06T06:04:21.419Z)

```

import asyncio
import aiosqlite
from datetime import datetime, UTC
from tg_video_filter import db
from tg_video_filter.models import Account, VideoRecord, ContentItem, Source

async def main():
    c = await aiosqlite.connect(":memory:")
    await db._apply_schema(c)
    await db.upsert_account(c, Account(id=111, telegram_id=111, created_at=datetime.now(UTC)))

    # Scenario A: insert a video with document_id in channel -100AAA, then look it up.
    v = VideoRecord(user_id=111, channel_id=-1001111111111, message_id=42, document_id=999888,
matched=True)
    await db.insert_video(c, v)

    got = await db.get_video_by_document_id(c, 111, 999888)
    print("A get_video_by_document_id:", got.channel_id if got else None, got.message_id if got
else None)

    # Scenario B: a content_item written by NON-shim code (Track C / future provider style)
    # with a non-numeric provider_message_id and a source whose provider_source_id is non-
numeric.
    # e.g. a WhatsApp-style source. Then list_videos_for_user projects ALL items for the
account.
    src = await db.upsert_source(c, Source(account_id=111, provider="whatsapp",
provider_source_id="group-abc", kind="group",

```

```

watched=True))
    item = ContentItem(account_id=111, source_id=src.id, provider="whatsapp",
                        provider_message_id="msg-xyz", content_type="video",
                        dedup_key="wa:msg:group-abc:msg-xyz", matched=True)
    await db.insert_content_item(c, item)
    try:
        vids = await db.list_videos_for_user(c, 111)
        print("B list_videos_for_user OK, count:", len(vids))
    except Exception as e:
        print("B list_videos_for_user RAISED:", type(e).__name__, e)

    # Scenario C: document_id whose value is non-numeric string stored via non-shim
    insert_content_item
    src2 = await db.upsert_source(c, Source(account_id=111, provider="telegram",
                                           provider_source_id="-100222222222", kind="channel",
watched=True))
    item2 = ContentItem(account_id=111, source_id=src2.id, provider="telegram",
                        provider_message_id="77", document_id="AGADabc", # non-numeric doc id
                        content_type="video", dedup_key="tg:doc:AGADabc", matched=True)
    await db.insert_content_item(c, item2)
    try:
        got2 = await db.get_video_by_document_id(c, 111, 12345) # unrelated numeric lookup
        print("C unrelated lookup:", got2)
    except Exception as e:
        print("C RAISED:", type(e).__name__, e)
    # get_video for that item's message
    try:
        got3 = await db.get_video(c, 111, -100222222222, 77)
        print("C get_video projection:", got3.document_id if got3 else None)
    except Exception as e:
        print("C get_video RAISED:", type(e).__name__, e)

    await c.close()

asyncio.run(main())

```

25. user (2026-07-06T06:04:21.420Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

26. user (2026-07-06T06:04:36.971Z)

```

A get_video_by_document_id: -100111111111 42
B list_videos_for_user RAISED: ValueError invalid literal for int() with base 10: 'group-abc'
C get_video RAISED: ValueError invalid literal for int() with base 10: 'AGADabc'

```

27. assistant (2026-07-06T06:04:55.965Z)

The repro confirms the ``int()`` casts raise on non-numeric values. But I must be honest about the scope: **is there any in-diff writer that produces non-numeric ``provider_source_id`` / ``document_id`` / ``provider_message_id``?** Within this PR's diff, the only writers are the shims (``insert_video``, ``upsert_channel``) and migration 003 ? all Telegram, all numeric. The non-numeric values in my repro came from directly calling ``insert_content_item`` with WhatsApp-style data, which no in-diff code does.

However, the PR **exports ``insert_content_item`` as public provider-neutral API** and **ships ``list_videos_for_user`` as the reader for ``/search``**. The provider-neutrality claim is that these tables support non-Telegram providers. The shims casting via ``int()`` are a **provider-neutrality leak**: the moment any non-Telegram content_item exists (which the schema explicitly supports ? ``provider`` column, ``whatsapp`` mentioned throughout), ``list_videos_for_user`` crashes for the whole account. But since no in-diff code creates such rows, and the prompt requires "defects that ship in THIS PR's diff" with a "concrete failure scenario (inputs -> wrong result)," I need to judge severity carefully. This is a **latent** leak, not yet triggerable by

shipped callers ? medium at most.

Let me now check the more concerning genuinely-triggerable-today issue. Let me re-examine the `document_id` uniqueness/collision and the `matched`-set-at-insert double-delivery across a restart during backfill more rigorously, plus check whether there's a within-Telegram scenario where `int()` fails. Telegram `document_ids` and `message_ids` are always integers. Channel ids always integers. So within Telegram, casts are safe.

Let me verify one more genuinely-triggerable concern: `get_video_by_content_fingerprint` returns an item whose source could be a `non-watched` or `different` source, and `int(src.provider_source_id)` ? always numeric for TG. Fine.

Now let me reconsider the `restart double-delivery` question one more time with fresh eyes, because the prompt emphasizes it. The claim in the code (line 1239) is "forwards are idempotent." Let me verify: on main, was `forwarded` ever used to gate? No. So the shim no-op changes nothing. But let me check: does the live path or backfill deliver based on `matched` for items `already in DB but not yet delivered`? No ? both paths `return`/`continue` on Tier-0 hit `before` evaluating/delivering. So an item inserted-but-not-delivered (e.g. process crashed between insert and forward) will `never be delivered` after restart ? on BOTH main and this branch. That's a pre-existing gap, not introduced here. Not a PR finding.

Let me double check the test file diffs for test_db to ensure no shipped-behavior defect I'm missing, and confirm ruff/format status already given. Let me verify the models change didn't drop the `forwarded` column handling in a way that breaks migration-002 rebuild.

28. user (2026-07-06T06:05:01.897Z)

```
src/tg_video_filter/db.py      | 802 ++++++-----
src/tg_video_filter/models.py | 112 +++++-
tests/test_db.py              | 519 ++++++-----
tests/test_delivery.py        | 16 +-
4 files changed, 1299 insertions(+), 150 deletions(-)
---DEDUP KEY TEST---
```

tg:doc:0
tg:fp:0:0:0

29. assistant (2026-07-06T06:05:13.947Z)

Interesting ? `document_id='0'` produces `tg:doc:0`. Let me check the guard: line 286 `if document_id is not None and document_id != ""`. So `'0'` is truthy-checked correctly (not None, not empty) ? `tg:doc:0`. Good, no degenerate collapse. And `insert_video` converts `int '0'` `document_id` to `str '0'`. But wait ? a real Telegram `document_id` is never 0. Fine.

Now the more interesting one: in `compute_dedup_key`, `document_id` is checked `!= ""`, but `insert_video` passes `str(video.document_id)` if `video.document_id` is not None else None. So a `document_id=0` (int) ? `'0'` ? `tg:doc:0`. Consistent. No bug.

Let me now verify the one genuinely-shippable concern I can find: whether `get_video_by_document_id` can return an item that is NOT actually the same document, due to the dedup_key lookup being loosely coupled. The dedup key `tg:doc:{id}` is exact. Fine.

Let me reconsider severity and whether the `int()` leak is truly unreachable in-diff. Let me check whether migration 003 could ever backfill a `non-numeric` `provider_source_id` or `document_id` from legacy `videos`. Legacy `channel_id`/`document_id`/`message_id` are `INTEGER` columns ? `CAST(... AS TEXT)` yields numeric strings. So migration is safe. Confirmed the leak is only reachable via the newly-exported `insert_content_item` / `upsert_source` with non-Telegram data, which nothing in-diff calls.

Let me do a final check: is there a `within-this-PR, within-Telegram` correctness bug in the shims? Let me test the `insert_video` ? `get_video` round trip for a video with `document_id=None` and partial metadata (Tier-0 fallback path), and a re-insert collision, to make sure the shim doesn't crash or misbehave for realistic v1 inputs.

30. user (2026-07-06T06:05:25.252Z)

```
inserted id: 1 channel: -100123 msg: 5 fwd: False
get_video: -100123 5 matched: True
re-insert: IntegrityError (expected)
list count: 1 chan: -100123 msg: 5
```

31. assistant (2026-07-06T06:05:34.387Z)

Telegram-path round trips work correctly. The shims are correct for all realistic in-diff (Telegram) inputs. The `int()` casts only fail for non-Telegram data that no in-diff code produces.

Let me make a final assessment of the two candidate findings:

1. `mark_forwarded` no-op / double-delivery** ? NOT a defect. `forwarded` was never read to gate delivery on main or branch; delivery is gated by Tier-0 `get_video` presence, and `insert_video` runs before `deliver`. Behavior is identical to main. The "forwards are idempotent" claim is irrelevant because re-delivery can't happen (Tier-0 skip). No finding.
2. `int()` provider-neutrality leak in `list_videos_for_user` / `_content_item_to_video` / `get_video_by_*` ? a real latent leak confirmed by repro, but not reachable by any in-diff caller (all shim/migration writers are Telegram-numeric). The schema and exported `insert_content_item` API explicitly invite non-Telegram rows, and the moment one exists, `list_videos_for_user` raises `ValueError` for the entire account. This ships in this PR's diff (the casts are new code). Severity: medium (latent; no in-diff trigger, but the PR's stated goal is provider-neutrality and these shims silently break it).

Let me verify the exact line numbers for the cast sites before reporting.

32. user (2026-07-06T06:05:35.835Z)

```
1112:         channel_id=int(src.provider_source_id),
1148:     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1173:     return _content_item_to_video(item, src, user_id, int(src.provider_source_id))
1261:         message_id=int(item.provider_message_id),
1262:         document_id=int(item.document_id) if item.document_id is not None else None,
1328:         channel_id=int(r["src_pid"]),
1329:         message_id=int(r["provider_message_id"]),
1330:         document_id=(int(r["document_id"]) if r["document_id"] is not None else None),
```

33. assistant (2026-07-06T06:05:43.668Z)

I have confirmed line numbers. Let me finalize. The single reportable finding is the provider-neutrality `int()` leak. I'll report it at medium severity, being honest that no in-diff caller triggers it today but the PR's exported API and schema make it reachable.

The `mark_forwarded` no-op is correctly reasoned to be a non-defect and I will not report it (it would be a false positive ? nothing gates on `forwarded`, so no double-delivery is introduced).

34. user (2026-07-06T06:05:57.615Z)

Structured output provided successfully